MACHINE LEARNING INTERVIEW PREPARATION

Classical NLP Foundations: Interview Questions & Answers

Comprehensive Classical NLP Foundations & Systems

Target Persona: Senior AI Engineer / Applied AI Engineer (~3 YOE)

Focus Areas: Text Preprocessing, Vector Spaces, BM25, Embeddings, Language Models, RNN/LSTM/GRU, Drift Detection

Includes: Step-by-Step Numerical Hand Calculations, PyTorch Implementations, Production Telemetry

Module 11: Interview Questions & Answers

This module provides detailed answers for the 40 standard and 10 advanced bonus interview questions covering NLP foundations, mathematical derivations, debugging procedures, and production systems design.

---

1. Conceptual Questions (1-12)

Question 1: What is NLP, and what are the major NLP tasks?

[ESSENTIAL]

Conversational Answer

I'd describe NLP as the branch of AI focused on getting computers to understand, interpret, and generate human language. The major task families are text classification — like sentiment analysis or spam detection — sequence tagging — like Named Entity Recognition and Part-of-Speech tagging — sequence-to-sequence generation — like translation and summarization — and question answering.

Intuitive Example

Feed a model the sentence "I loved the movie but hated the ending" and a classifier outputs a single label (mixed sentiment), a tagger labels each word's grammatical role, and a generator could rewrite it as a one-line review summary. Same input text, three completely different task shapes.

Key Interview Points

- **Text Classification**: Whole-document label assignment.
- **Sequence Tagging**: Per-token label assignment (NER, POS).
- **Sequence-to-Sequence Generation**: Producing new token sequences (translation, summarization).

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

There's no single governing formula for "NLP" as a field — it's a task taxonomy, not one algorithm. The unifying idea is that every task ultimately reduces to mapping a token sequence to either a label, a sequence of labels, or another token sequence.

Production Perspective & Trade-offs

Different task families carry very different latency budgets. Classification and tagging are typically single forward passes and can hit sub-50ms latency with sparse or small models. Autoregressive generation requires one forward pass *per output token*, so its latency scales with output length and is the most expensive task family to serve at scale.

Common Mistakes

1. Treating sequence tagging (NER) and sequence-to-sequence generation (translation) as the same task shape — they have different output structures and different model heads.
2. Assuming a single model architecture is "best for NLP" rather than matching architecture to task shape and latency budget.

COMMON FOLLOW-UP QUESTIONS

> **Q: What makes NER harder than text classification?**
>
> NER requires token-level context and boundary detection (where does the entity start/end), while classification only needs a single pooled representation of the whole sequence.

> **Q: Which task family is hardest to serve at low latency in production?**
>
> Sequence-to-sequence generation, because autoregressive decoding is inherently sequential — you can't parallelize across output tokens the way you can across a classification batch.

One-Line Takeaway

> **Takeaway:** NLP is a task taxonomy (classification, tagging, generation, QA), and the task shape — not "NLP" as a whole — determines the model architecture and latency budget you need.

Question 2: Explain a typical NLP pipeline from raw text to prediction.

[ESSENTIAL]

Conversational Answer

The standard pipeline moves through: raw input ingestion, then text cleaning (normalization, regex), then tokenization (splitting into subwords), then representation (mapping tokens to sparse or dense vectors), then model execution (recurrent or self-attention layers), then a prediction output (logits), and finally metric evaluation.

Intuitive Example

The string "Seattle's libraries are awesome!" gets lowercased and stripped of punctuation, split into subword tokens, mapped to embedding indices, run through a model, and turned into a prediction like "Sentiment: POSITIVE (98.4%)" — each stage narrows unstructured text down to a structured numerical decision.

Key Interview Points

- Preprocessing & normalization
- Subword tokenization
- Vector embeddings
- Inference execution

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Each stage is a deterministic transformation of the previous stage's output shape: text → cleaned text → token IDs `[L]` → embeddings `[L, d]` → model output (logits). The pipeline is only as correct as its weakest, least-tested stage — usually preprocessing.

Production Perspective & Trade-offs

Preprocessing steps must be byte-for-byte identical between training and inference to avoid training-serving skew. Real-time inference pipelines typically cache embedding weight lookups in memory to avoid repeated database round-trips.

Common Mistakes

1. Ignoring Unicode normalization, which leads to split character representations between training and serving.
2. Letting training and serving preprocessing code diverge into separate codebases without shared tests.

COMMON FOLLOW-UP QUESTIONS

> **Q: Where does training-serving skew typically occur in this pipeline?**
>
> Usually in the normalization step — differences in lowercasing rules or Unicode normalization between the training and serving code paths.

> **Q: Why cache embedding lookups in production?**
>
> Because repeated database or disk lookups for the same frequent tokens add avoidable latency; an in-memory cache turns that into a fast array index.

One-Line Takeaway

> **Takeaway:** The NLP pipeline is a chain of shape transformations from raw text to logits, and it breaks silently — not loudly — when preprocessing drifts between training and serving.

Question 3: What is the difference between stemming and lemmatization?

[ESSENTIAL]

Conversational Answer

Stemming is a rule-based heuristic that chops word endings off to approximate a base form — fast, but it can produce non-words. Lemmatization uses a morphological dictionary lookup plus part-of-speech context to resolve a word to its real dictionary root.

Intuitive Example

"studies" stems to "studi" (not a real word) but lemmatizes to "study". "wolves" stems to "wolv" but lemmatizes to "wolf" — lemmatization costs more but always returns something you could look up in a dictionary.

Key Interview Points

- Heuristic suffix truncation (Stemming)
- Morphological dictionary lookup (Lemmatization)
- POS-tagging dependency for lemmatization

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

No formula is required here — the distinction is algorithmic (regex-style suffix rules vs. dictionary lookup keyed by POS tag), not mathematical.

	Stemming	Lemmatization
Method	Rule-based suffix truncation	Dictionary lookup + POS context
Output validity	Can produce non-words	Always a valid dictionary root

	Stemming	Lemmatization
Speed	Very fast, $O(L)$ per token	Slower, requires POS tags + lookup index
Memory	~0 (rules only)	Morphology DB loaded in RAM (10-50MB)
Best for	High-throughput indexing (search engines)	Semantic accuracy (QA, semantic search)

Production Perspective & Trade-offs

Stemming is integrated directly into web-scale indexing pipelines because it's essentially free. Lemmatization requires an in-memory morphology database and a POS tagger pass first, which is why modern neural pipelines increasingly skip both and let subword tokenization + embedding learning absorb root-mapping instead.

Common Mistakes

1. Assuming stemming is always superior because of its lower latency — it can hurt semantic tasks by mangling words.
2. Feeding a lemmatizer the wrong POS tag (e.g. tagging "saw" as a noun instead of a verb), which silently produces the wrong lemma.

COMMON FOLLOW-UP QUESTIONS

> **Q: Which is preferred for web search indexing?**
>
> Lemmatization is generally preferred when index quality matters, because it maps words to real dictionary terms and improves recall on morphological variants — though many production search indexers still use stemming purely for its speed.

> **Q: Why does POS tagging accuracy matter for lemmatization?**
>
> The lemmatizer's dictionary lookup is keyed by both the word and its POS tag; a wrong tag routes the lookup to the wrong sense of the word and returns an incorrect lemma.

One-Line Takeaway

> **Takeaway:** Stemming trades correctness for speed via suffix rules; lemmatization trades speed for a real dictionary root via POS-aware lookup.

Question 4: Why is tokenization necessary?

[ESSENTIAL]

Conversational Answer

Models can't operate on raw strings — they need discrete units mapped to numeric indices. Tokenization decomposes text into words, subwords, or characters so each unit can be looked up in a fixed vocabulary table and converted into an embedding vector.

Intuitive Example

The string "don't" isn't naturally one unit — a naive whitespace split would keep it as one token and miss that it's really "do" + "not"; a proper tokenizer treats the apostrophe as a boundary and produces cleaner, more generalizable sub-units.

Key Interview Points

- Lexical splitting
- Vocabulary index mapping
- Index lookup tables for embeddings

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Tokenization is the boundary decision that fixes both vocabulary size V and sequence length L for everything downstream — every other design choice (embedding table size, attention cost) inherits from this one decision.

Production Perspective & Trade-offs

A large vocabulary increases the size of the model's input/output projection layers ($V \times d$ parameters); a small, character-heavy vocabulary keeps that projection small but inflates sequence length L, which raises attention computation cost quadratically ($O(L^2)$).

Common Mistakes

1. Treating tokenization as a simple whitespace split, which fails on punctuation and clitics like "don't".
2. Not accounting for how the vocabulary size choice ripples into both embedding memory and attention compute cost.

COMMON FOLLOW-UP QUESTIONS

> **Q: Can we build an NLP system without a tokenizer?**
>
> Yes — byte-level models process raw bytes directly, avoiding the tokenizer entirely, but at the cost of much longer sequences and slower training.

> **Q: What's the practical effect of choosing too large a vocabulary?**
>
> It bloats the embedding and output projection layers, directly increasing VRAM footprint and load time, often for diminishing returns past a few hundred thousand tokens.

One-Line Takeaway

> **Takeaway:** Tokenization is the boundary decision that trades vocabulary size against sequence length, and that trade-off propagates into every downstream memory and compute cost.

Question 5: Compare character, word, and subword tokenization.

[ESSENTIAL]

Conversational Answer

Character tokenization keeps the vocabulary tiny but makes sequences very long. Word tokenization keeps sequences short but the vocabulary can explode into the millions, with high out-of-vocabulary rates. Subword tokenization — BPE or WordPiece — splits common roots into single tokens and decomposes rare words into pieces, landing in the middle.

Intuitive Example

"unfriendliness" as characters is 14 tokens; as a whole word it's one token (if it's even in the vocabulary — likely not); as subwords it might be 3 tokens: "un", "friend", "liness" — short, and every piece is a token the model has actually seen before.

Key Interview Points

- Vocabulary size vs. sequence length trade-off
- Out-of-Vocabulary (OOV) rate
- Computational balance point

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

No single formula governs the choice — it's a direct trade-off between V (vocabulary size, drives embedding memory) and L (sequence length, drives attention compute via $O(L^2)$).

	Character	Word	Subword (BPE/WordPiece)
Vocabulary size	Tiny (~100s)	Massive (millions), high OOV	Moderate (32k-256k)
Sequence length	Very long	Short	Moderate
OOV handling	None needed	Falls back to `<unk>`	Decomposes into known pieces
Production standard	Rare	Legacy / classical NLP	Standard in modern LLMs

Production Perspective & Trade-offs

Subword tokenizers are the production standard for LLMs precisely because they keep the vocabulary table compact (32k-256k) while eliminating catastrophic OOV failures on typos, rare names, and new terms.

Common Mistakes

1. Believing character-level tokenization is more computationally efficient — it actually increases sequence length, which raises attention cost quadratically.
2. Assuming word-level tokenization is "simpler" without accounting for its OOV failure mode in production.

COMMON FOLLOW-UP QUESTIONS

> **Q: Which tokenizer type is most robust against typos?**
>
> Character-level or subword tokenizers, since they can decompose a misspelled word into known character sequences instead of failing outright.

> **Q: Why don't production LLMs use pure word-level tokenization anymore?**
>
> The vocabulary would need to be effectively unbounded to avoid OOV failures, which is computationally infeasible at the embedding-layer level.

One-Line Takeaway

> **Takeaway:** Character tokenization minimizes vocabulary at the cost of sequence length; word tokenization does the reverse; subword tokenization is the balance point production systems converge on.

Question 6: Why did modern NLP move from word tokenization to subword tokenization?

[ESSENTIAL]

Conversational Answer

Word-level tokenization runs into a vocabulary explosion problem — past a million unique tokens — and a high out-of-vocabulary rate on anything not seen during training. Subword tokenization represents text with a much smaller set of statistically learned roots, prefixes, and suffixes, so new or misspelled words can still be decomposed instead of falling back to `<unk>`.

Intuitive Example

A word-level model trained without ever seeing "tokenization" would map it to `<unk>` and lose all signal. A subword model instead splits it into pieces like "token" + "ization", both of which it has seen many times, and keeps meaningful signal intact.

Key Interview Points

- Vocabulary explosion at word level
- Out-of-Vocabulary (OOV) mitigation
- Statistically learned decomposition, not linguist-defined

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Subword vocabularies are built by frequency-driven merge algorithms (BPE) or likelihood-driven scoring (WordPiece) — the vocabulary is *learned* from the training corpus, not hand-authored.

Production Perspective & Trade-offs

A compact subword vocabulary fits comfortably in GPU memory, freeing VRAM budget for longer context windows or larger hidden dimensions instead of an oversized embedding table.

Common Mistakes

1. Believing subwords are manually defined by linguists — they're learned statistically from corpus frequency or likelihood.
2. Assuming a bigger vocabulary is strictly better — it trades VRAM for shorter sequences, and that trade-off has diminishing returns.

COMMON FOLLOW-UP QUESTIONS

> **Q: How does BPE handle numerical data?**
>
> It typically splits numbers into individual digits or short byte sequences, avoiding the need for a dedicated token per integer.

> **Q: Does a larger subword vocabulary always improve model quality?**
>
> Not necessarily — beyond a certain size the sequence-length savings plateau while embedding-table memory keeps growing, so there's a practical sweet spot rather than "bigger is better."

One-Line Takeaway

> **Takeaway:** Subword tokenization replaced word-level tokenization because it caps vocabulary size while still statistically decomposing any unseen word instead of discarding it as `<unk>`.

Question 7: What are Out-of-Vocabulary (OOV) words, and how can they be handled?

[ESSENTIAL]

Conversational Answer

OOV words are tokens seen at inference time that weren't in the training vocabulary. The standard fixes are: fall back to an `<unk>` token (loses information), decompose the word with a subword tokenizer like BPE (keeps partial signal), or reconstruct a vector from character n-grams the way FastText does.

Intuitive Example

If "cryptocurrency" wasn't in a word-level model's vocabulary in 2010, it would map straight to `<unk>` and vanish. A subword tokenizer instead splits it into pieces like "crypto" + "currency" that carry real signal even for a word the model never saw whole.

Key Interview Points

- Unknown token fallback (`<unk>`)

- Byte-fallback / subword decomposition
- FastText n-gram reconstruction

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

FastText's approach is additive: a word's vector is the sum of its character n-gram vectors, $\mathbf{v}_w = \sum_{g \in \mathcal{G}_w} \mathbf{z}_g$ — so even an unseen word gets a vector as long as its subword pieces were seen during training.

Production Perspective & Trade-offs

Falling back to `<unk>` degrades accuracy because all semantic content of that token is lost. Subword and n-gram based fallbacks preserve partial signal at the cost of slightly longer sequences (subwords) or larger models (FastText's n-gram bucket tables).

Common Mistakes

1. Believing that growing the training vocabulary to cover "every possible word" solves OOV — it doesn't scale and new words keep appearing after deployment.
2. Forgetting that `<unk>` fallback silently destroys information rather than failing loudly.

COMMON FOLLOW-UP QUESTIONS

> **Q: Why doesn't BERT return OOV errors?**
>
> BERT uses WordPiece tokenization, which decomposes an unrecognized word down to smaller known subwords (or individual characters as a last resort) instead of ever emitting ``.

> **Q: What's the trade-off of FastText's n-gram approach vs. subword tokenization?**
>
> FastText reconstructs OOV vectors via summed n-gram embeddings, which needs a large hash-bucket table in memory; subword tokenization instead avoids OOV entirely by construction, at the cost of longer sequences for rare words.

One-Line Takeaway

> **Takeaway:** OOV is an unavoidable consequence of any fixed vocabulary — the practical question is whether your fallback (`<unk>` , subwords, or n-grams) discards signal or preserves it.

Question 8: Explain the Distributional Hypothesis.

[ESSENTIAL]

Conversational Answer

The Distributional Hypothesis says that words appearing in similar contexts tend to share meaning. It's the foundational assumption behind every dense word embedding method — you learn a word's vector by predicting or counting its neighbors, not by consulting a dictionary.

Intuitive Example

"I drank a glass of ___" tends to be completed by "water", "juice", or "milk" — words that never even need to co-occur with each other directly still end up with similar vectors because they share the same surrounding contexts.

Key Interview Points

- Contextual semantics from co-occurrence
- No manual semantic labeling required
- Foundation of Word2Vec, GloVe, and contextual embeddings alike

---

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

No formula is needed to state the hypothesis itself — it's the assumption that licenses every count-based or predictive embedding objective downstream (Word2Vec's context prediction, GloVe's co-occurrence factorization).

Production Perspective & Trade-offs

Because the hypothesis lets embeddings be learned from raw, unlabeled text, it eliminates the need for expensive manual semantic annotation — this is precisely why embedding pretraining scales to web-sized corpora.

Common Mistakes

1. Assuming the hypothesis requires dictionary-defined semantic tags — it only requires raw co-occurrence statistics.
2. Forgetting its known failure mode: antonyms.

COMMON FOLLOW-UP QUESTIONS

Q: What is a key limitation of this hypothesis?

Antonyms like "hot" and "cold" frequently appear in nearly identical contexts, so distributional methods often assign them very similar vectors despite opposite meanings.

Q: Does the Distributional Hypothesis apply to contextual embeddings (BERT) too?

Yes — self-attention is itself a context-aggregation mechanism, so contextual embeddings are a direct, more expressive extension of the same underlying assumption.

One-Line Takeaway

> **Takeaway:** The Distributional Hypothesis — "context determines meaning" — is the unlabeled-data assumption that makes every embedding method, static or contextual, possible.

Question 9: Why do sparse text representations fail to capture semantic similarity?

[ESSENTIAL]

Conversational Answer

Sparse representations like one-hot vectors or Bag-of-Words treat every word as its own independent, orthogonal dimension. That means the dot product between two related words — "cat" and "feline" — is exactly zero, so there's no way for the representation itself to express that they're similar.

Intuitive Example

In a one-hot vocabulary space, "cat" is the vector $[1, 0, 0, \dots]$ and "feline" is $[0, 1, 0, \dots]$ — mathematically as unrelated as "cat" and "spreadsheet", because orthogonality carries no notion of meaning.

Key Interview Points

- Vector orthogonality
- Sparse, high-dimensional representation
- Zero dot-product between synonyms

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Cosine similarity between any two distinct one-hot vectors is always 0 by construction — there's no shared dimension for the metric to detect, regardless of true semantic closeness.

Production Perspective & Trade-offs

Sparse representations scale directly with vocabulary size $|V|$, producing high memory footprint and severe data sparsity for downstream models — this was a primary motivation for moving to dense embeddings.

Common Mistakes

1. Assuming TF-IDF captures word similarity — it only matches exact token strings, with the same orthogonality problem as raw Bag-of-Words.
2. Conflating "sparse representation" with "bad representation" — sparse methods (BM25) still dominate first-stage retrieval precisely because they're fast and interpretable, just not semantically aware.

COMMON FOLLOW-UP QUESTIONS

> **Q: How does Cosine Similarity help evaluate sparse vectors at all?**
>
> It measures overlap on shared coordinates (shared exact terms), which works for keyword matching but still fails whenever the compared documents use synonyms instead of the same words.

> **Q: What's the direct fix for this limitation?**
>
> Move from sparse, orthogonal representations to dense embeddings (Word2Vec, GloVe, or contextual models), where semantic similarity is encoded as geometric proximity instead of exact-match overlap.

One-Line Takeaway

> **Takeaway:** Sparse representations are semantically blind by construction — every distinct word is mathematically orthogonal to every other, synonyms included.

Question 10: Compare static embeddings and contextual embeddings.

[ESSENTIAL]

Conversational Answer

Static embeddings like Word2Vec or GloVe give every word exactly one vector, no matter the context it appears in. Contextual embeddings, from models like BERT or GPT, generate a *different* vector for the same word depending on the sentence around it — so they can tell the difference between "bank" in "river bank" and "bank" in "money bank."

Intuitive Example

Look up "bank" in a static embedding table and you get back one fixed 300-dimensional vector, always. Run "bank" through BERT in two different sentences and you get two different vectors, because the surrounding words shape the representation at inference time.

Key Interview Points

- Polysemy resolution
- Static vocabulary table, $O(1)$ lookup
- Dynamic encoder projection per occurrence

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Static lookup is literally an indexing operation into a $[V, d]$ matrix. Contextual embeddings instead run self-attention over the full input sequence, so each token's output vector is a function of every other token in the sequence, not just the token identity itself.

	Static (Word2Vec / GloVe)	Contextual (BERT / GPT)
Vector per word	Exactly one, fixed	One per occurrence, context-dependent
Polysemy handling	None — same vector regardless of sense	Resolves sense from surrounding context
Inference cost	$O(1)$ table lookup	Full transformer forward pass
Typical use case	Low-latency similarity search, cold-start features	Classification, QA, semantic search re-ranking

Production Perspective & Trade-offs

Contextual embeddings require a transformer forward pass per input, which is orders of magnitude more expensive than a table lookup — this matters directly for serving cost and latency budgets. Static embeddings remain useful precisely because they're cheap: they're still common as a fast first-pass signal or as a fallback when GPU budget is constrained.

Common Mistakes

1. Assuming static embeddings are simply obsolete — they're still the right choice when latency or cost rules out a transformer forward pass.
2. Forgetting that contextual embeddings change between runs/positions — you cannot cache a single vector per word the way you can with static embeddings.

COMMON FOLLOW-UP QUESTIONS

> **Q: Can static embeddings resolve part-of-speech ambiguity?**
>
> No — the vector for a word is identical whether it's used as a noun or a verb, since it's a fixed per-word lookup with no sentence context.

> **Q: Why is BERT's contextual embedding for the same word different across two sentences?**
>
> Because self-attention lets every token's representation absorb information from every other token in that specific sequence, so the same word identity produces a different output vector depending on its neighbors.

One-Line Takeaway

> **Takeaway:** Static embeddings trade context-sensitivity for $O(1)$ lookup speed; contextual embeddings trade inference cost for polysemy resolution — pick based on your latency budget, not just accuracy.

Question 11: Why were RNNs introduced after Bag-of-Words and TF-IDF?

[ESSENTIAL]

Conversational Answer

Bag-of-Words and TF-IDF discard word order entirely — "dog bites man" and "man bites dog" produce the same vector. RNNs process tokens sequentially and carry a hidden state forward, so they can capture word order and temporal dependencies that count-based methods structurally cannot represent.

Intuitive Example

TF-IDF sees "not good" and "good, not bad" as bags containing the same words with no notion of order; an RNN reads them left to right and updates its hidden state differently at each step, so it can in principle tell the two apart.

Key Interview Points

- Sequence order preservation
- Variable-length input handling
- Hidden state as running memory

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

The RNN hidden state update $\mathbf{h}_t = \tanh(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t + \mathbf{b}_h)$ makes $\mathbf{h}_t$ a function of the entire prefix $x_1, \ldots, x_t$, not just the current token — this is exactly the order-sensitivity that Bag-of-Words/TF-IDF lack.

Production Perspective & Trade-offs

The same recurrence that gives RNNs order-sensitivity also creates a training bottleneck: because $\mathbf{h}_t$ depends on $\mathbf{h}_{t-1}$, the sequence dimension cannot be parallelized on GPU the way a batch dimension can.

Common Mistakes

1. Believing RNNs can process all tokens of a sequence in parallel like Transformers — the recurrence is inherently sequential.
2. Forgetting that larger hidden state sizes increase VRAM usage during Backpropagation Through Time (BPTT), since every intermediate hidden state must be kept for the backward pass.

COMMON FOLLOW-UP QUESTIONS

> **Q: How does RNN hidden state size impact VRAM usage during training?**
>
> BPTT must retain every intermediate hidden state across all L time steps to compute gradients, so VRAM usage scales with both hidden dimension and sequence length, not just hidden dimension alone.

> **Q: What specifically do RNNs add that Bag-of-Words structurally cannot represent?**
>
> Order and positional dependency — Bag-of-Words vectors are permutation-invariant by construction, while an RNN's hidden state is a function of token order.

One-Line Takeaway

> **Takeaway:** RNNs were introduced to recover word order, which Bag-of-Words and TF-IDF discard by design — at the cost of a sequential architecture that can't parallelize across time steps.

Question 12: Why were Transformers able to replace RNNs?

[ESSENTIAL]

Conversational Answer

Transformers replaced RNNs by using self-attention instead of recurrence — every token attends to every other token directly, so the path length between any two tokens is $O(1)$ instead of $O(L)$, and the whole sequence can be processed in parallel during training.

Intuitive Example

In an RNN, information from token 1 has to pass through every intermediate hidden state to reach token 100 — a long, sequential relay. In a Transformer, token 100 can attend directly to token 1 in a single attention operation, and every token's attention computation happens simultaneously.

Key Interview Points

- Parallel training across the sequence dimension
- $O(1)$ path length vs. RNN's $O(L)$
- Self-attention as the core mechanism

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Self-attention computes an $L \times L$ score matrix, giving every token pair a direct connection ($O(1)$ path length) at the cost of $O(L^2)$ time and memory — a fundamentally different trade-off surface than an RNN's $O(L)$ sequential steps with $O(1)$ per-step memory.

Production Perspective & Trade-offs

Transformers train much faster than RNNs because the sequence dimension parallelizes across GPU cores, but that same $O(L^2)$ attention matrix makes serving very long sequences memory-intensive — the opposite bottleneck from RNNs, which are slow to train but cheap in per-step memory.

Common Mistakes

1. Assuming Transformers always use less memory than RNNs — their attention matrix scales quadratically with sequence length and can be the dominant memory cost for long contexts.
2. Forgetting that self-attention is permutation-invariant on its own and requires positional encodings to recover order information.

COMMON FOLLOW-UP QUESTIONS

> **Q: Why do Transformers need positional encodings?**
>
> Self-attention treats the input as an unordered set of tokens by default; without positional encodings it has no way to distinguish "dog bites man" from "man bites dog."

> **Q: At what point does a Transformer's memory cost exceed an RNN's for a given sequence?**
>
> Once sequence length L grows large enough that the $O(L^2)$ attention matrix outweighs the RNN's linear $O(L)$ memory footprint — the crossover point depends on hidden dimension d and available VRAM.

One-Line Takeaway

> **Takeaway:** Transformers replaced RNNs by trading recurrence's cheap-but-sequential $O(L)$ path length for attention's parallel-but-quadratic $O(1)$ path length.

---

2. Mathematical Questions (13-22)

Question 13: Derive the TF-IDF equation and compute TF-IDF scores for a small corpus.

[ESSENTIAL]

Conversational Answer

TF-IDF scores a term by multiplying how often it appears in *this* document (Term Frequency) by how rare it is *across the whole corpus* (Inverse Document Frequency) — so a word that's frequent locally but rare globally scores highest, while universal words like "the" get suppressed.

Intuitive Example

In a two-document corpus — "cat mat" and "mat rug" — "mat" appears in both documents so it gets a low IDF (it's not distinctive), while "cat" and "rug" each appear in only one document so they get a higher IDF and stand out as the more informative terms.

Key Interview Points

- **Term Frequency (TF)**: How often a term appears in a document.
- **Inverse Document Frequency (IDF)**: How rare a term is across the corpus.
- L_2 **Normalization**: Removes document-length bias.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

$$\text{TF-IDF}(t, d, D) = \text{TF}(t, d) \times \text{IDF}(t, D), \qquad \text{IDF}(t, D) = \ln\left(\frac{1+N}{1+\text{DF}(t)}\right) + 1$$

For the micro-corpus $d_1 =$ "cat mat", $d_2 =$ "mat rug" ($N = 2$, vocabulary $\{$cat, mat, rug$\}$): $\text{DF}(\text{cat}) = 1 \Rightarrow \text{IDF} \approx 1.4055$; $\text{DF}(\text{mat}) = 2 \Rightarrow \text{IDF} = 1.0$. The raw TF-IDF vector for d_1 is $[1.4055, 1.0, 0.0]$, which after L_2 normalization becomes $\approx [0.8148, 0.5797, 0.0]$.

Production Perspective & Trade-offs

L_2 normalization is essential because longer documents naturally repeat words more, inflating raw TF scores — normalizing onto a unit hypersphere lets Cosine Similarity measure angle (topic alignment) rather than raw magnitude (document length). At scale ($|V| > 100{,}000$), TF-IDF matrices are stored in sparse formats (COO/CSR) to avoid wasting memory on the overwhelming majority of zero entries.

Common Mistakes

1. Forgetting to apply smoothing or normalization, which introduces document-length bias into similarity comparisons.
2. Computing IDF without the $+1$ smoothing terms, which breaks on a document frequency of zero.

COMMON FOLLOW-UP QUESTIONS

> **Q: How does a document frequency of 0 impact IDF?**
>
> The smoothed IDF formula adds 1 to both the numerator and denominator specifically to prevent a division-by-zero or undefined-log error for terms that don't appear in any document of a reference set.

> **Q: Why store TF-IDF matrices in sparse format at scale?**
>
> Because any single document only contains a tiny fraction of the full vocabulary, a dense matrix wastes enormous memory on zeros — CSR/COO formats store only the non-zero entries and their indices.

One-Line Takeaway

> **Takeaway:** TF-IDF rewards terms that are frequent locally but rare globally, and L_2 normalization is what makes the resulting vectors comparable regardless of document length.

Question 14: Compute cosine similarity between two TF-IDF vectors.

[ESSENTIAL]

Conversational Answer

Cosine similarity measures the angle between two vectors rather than their raw magnitude, which is exactly what you want when comparing documents of different lengths — it tells you how aligned their term distributions are, not which one has more words.

Intuitive Example

Given $\mathbf{a} = [0.8, 0.6, 0.0]$ and $\mathbf{b} = [0.0, 0.6, 0.8]$, only the middle dimension overlaps, so the dot product is 0.36 — a moderate similarity driven entirely by the one shared term.

Key Interview Points

- Angle-based similarity, not magnitude-based
- Simplifies to a dot product when vectors are pre-normalized
- Standard for sparse and dense vector comparison alike

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

$$\text{CosineSimilarity}(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{||\mathbf{a}||_2 ||\mathbf{b}||_2}$$

If both vectors are already L_2-normalized, the denominator is 1 and cosine similarity reduces to a plain dot product.

Production Perspective & Trade-offs

Dot products run extremely efficiently on modern hardware via BLAS libraries and vector databases' ANN indexes, which is why cosine similarity (or its normalized dot-product form) is the default metric for large-scale semantic search.

Common Mistakes

1. Neglecting to normalize vectors before comparing raw dot products, which reintroduces a length bias.
2. Confusing cosine similarity (bounded, angle-based) with Euclidean distance (unbounded, magnitude-sensitive) — they rank differently on unnormalized vectors.

COMMON FOLLOW-UP QUESTIONS

> **Q: What is the cosine similarity of two orthogonal vectors?**
>
> Exactly 0, since they share no overlapping non-zero dimensions and their dot product is zero.

> **Q: Why do vector databases pre-normalize embeddings at index time?**
>
> So that similarity search can use a plain (cheaper) dot product at query time instead of computing norms repeatedly for every comparison.

One-Line Takeaway

> **Takeaway:** Cosine similarity measures directional alignment, not magnitude, which is exactly the length-invariance property you need when comparing documents of different sizes.

Question 15: Using the chain rule, derive the probability of a sentence in an N-gram language model.

[ESSENTIAL]

Conversational Answer

The chain rule lets you factor the joint probability of a whole sentence into a product of conditional probabilities, one token at a time. An N-gram model then simplifies each conditional by only looking back at the previous $N-1$ words instead of the full history — that's the Markov assumption.

Intuitive Example

Scoring "the cat sat" doesn't require the joint probability of all three words at once — it's broken into $P(\text{the}) \times P(\text{cat} \mid \text{the}) \times P(\text{sat} \mid \text{the}, \text{cat})$, and a bigram model would further truncate the last term to just $P(\text{sat} \mid \text{cat})$.

Key Interview Points

- Joint probability factorization via the chain rule
- Markov assumption truncates the context window

- Trade-off between context length and data sparsity

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

$$P(w_1, \ldots, w_m) = \prod_{i=1}^{m} P(w_i \mid w_1, \ldots, w_{i-1}) \approx \prod_{i=1}^{m} P(w_i \mid w_{i-N+1}, \ldots, w_{i-1})$$

The first form is exact but intractable (infinite possible histories); the second is the N-gram approximation that makes estimation from finite corpus counts feasible.

Production Perspective & Trade-offs

Larger context orders N capture more dependencies but blow up the table memory footprint exponentially, $O(|V|^N)$ — this is precisely why production systems rarely go past trigrams before switching to neural sequence models entirely.

Common Mistakes

1. Forgetting boundary markers (`<s>`, `</s>`) when computing sequence probabilities, which breaks the factorization at sentence edges.
2. Multiplying raw probabilities directly in code, causing numerical underflow on longer sequences.

COMMON FOLLOW-UP QUESTIONS

> **Q: Why use log probabilities in evaluation code?**
> Multiplying many small probabilities together underflows to zero in floating point; summing their logs instead keeps the computation numerically stable.

> **Q: What's lost by truncating to an N-gram approximation?**
> Any dependency spanning more than $N - 1$ tokens back — for example, a bigram model can't verify that a closing parenthesis matches one opened many tokens earlier.

One-Line Takeaway

> **Takeaway:** The chain rule gives the exact joint sequence probability; the Markov assumption is what makes estimating it from finite data tractable, at the cost of a bounded context window.

Question 16: Explain Maximum Likelihood Estimation (MLE) for N-gram language models.

[ESSENTIAL]

Conversational Answer

MLE estimates a bigram transition probability by directly counting: how often did this pair of words co-occur, divided by how often the first word appeared at all. It's the simplest possible estimator, and it's also exactly why N-gram models break on anything unseen.

Intuitive Example

If "sat on" appears 10 times in a corpus and "sat" appears 100 times total, MLE estimates $P(\text{on} \mid \text{sat}) = 10/100 = 0.1$ — a direct frequency ratio, no smoothing involved.

Key Interview Points

- Direct frequency-ratio estimator
- $O(1)$ lookup at inference with precomputed tables
- Breaks completely on unseen prefixes

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

$$P_{\text{MLE}}(w_i \mid w_{i-1}) = \frac{C(w_{i-1}, w_i)}{C(w_{i-1})}$$

This is an unsmoothed ratio of corpus counts — nothing more.

Production Perspective & Trade-offs

MLE lookups are $O(1)$ if precomputed into a hash table, making them extremely cheap at inference time, but the memory footprint of the count table itself scales as $O(|V|^N)$ for an N-gram model, which is the real production constraint.

Common Mistakes

1. Assuming MLE generalizes to unseen text without smoothing — any zero-count transition collapses the entire sequence probability to zero.
2. Confusing "cheap at inference" with "cheap overall" — the count table itself is the expensive part to build and store.

COMMON FOLLOW-UP QUESTIONS

> **Q: What happens if a prefix is unseen during training?**
>
> The denominator $C(w_{i-1})$ becomes zero, making the ratio undefined — this is exactly the failure mode that smoothing techniques exist to fix.

> **Q: Why is MLE described as "unbiased but brittle"?**
>
> It's the statistically correct estimator given the observed counts, but it assigns zero probability to anything not observed, which is far too brittle for real-world text with a long tail of rare combinations.

One-Line Takeaway

> **Takeaway:** MLE is just a corpus count ratio — correct on what it's seen, and completely undefined on what it hasn't, which is why it's never used unsmoothed in production.

Question 17: Why is Laplace smoothing required? Compute smoothed probabilities for a simple example.

[ESSENTIAL]

Conversational Answer

Under raw MLE, any unseen word transition gets probability zero, and because the chain rule multiplies probabilities together, a single zero collapses the entire sentence's probability to zero. Laplace smoothing fixes this by adding 1 to every count, so no transition — seen or unseen — ever gets exactly zero probability.

Intuitive Example

Training on "the cat sat on the mat," the transition "the" → "sat" was never observed. Raw MLE would give it probability 0; Laplace smoothing instead gives it a small but non-zero probability of $1/7 \approx 0.1429$, keeping the model usable on new text.

Key Interview Points

- Add-one smoothing prevents zero-probability collapse
- Reallocates probability mass from seen to unseen events
- Simple but crude at scale

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

$$P_{\text{Laplace}}(w_i \mid w_{i-1}) = \frac{C(w_{i-1}, w_i) + 1}{C(w_{i-1}) + |V|}$$

Training on "the cat sat on the mat" ($|V| = 5$): $P_{\text{Laplace}}(\text{cat} \mid \text{the}) = \frac{1+1}{2+5} \approx 0.2857$, while the unseen pair $P_{\text{Laplace}}(\text{sat} \mid \text{the}) = \frac{0+1}{2+5} \approx 0.1429$ — non-zero purely because of the $+1$ smoothing term.

Production Perspective & Trade-offs

Laplace smoothing guarantees mathematical validity, but for large vocabularies ($|V| = 50{,}000$) it allocates a disproportionate amount of probability mass to the long tail of rare and unseen combinations, degrading the probability of common, natural transitions. Production systems use Kneser-Ney or absolute discounting instead, which discount a fixed amount from seen counts and redistribute it based on how likely a word is to complete a novel context.

Common Mistakes

1. Forgetting to add $|V|$ to the denominator, which breaks probability normalization.
2. Using Laplace smoothing in production at scale, where its crude uniform redistribution noticeably degrades common-case accuracy.

COMMON FOLLOW-UP QUESTIONS

> **Q: How does Kneser-Ney smoothing improve on Laplace?**
>
> It discounts a fixed amount from observed counts and redistributes it based on a word's *continuation probability* — how likely it is to appear after novel contexts — rather than uniformly across the whole vocabulary.

> **Q: Why is add-one smoothing considered "too aggressive" for large vocabularies?**
>
> Because it treats every one of the $|V|$ possible unseen transitions as equally likely, which spreads probability mass far too thin when $|V|$ is in the tens of thousands.

One-Line Takeaway

> **Takeaway:** Laplace smoothing trades mathematical validity (no more zero probabilities) for a crude uniform redistribution that production systems replace with Kneser-Ney at scale.

Question 18: What is Perplexity? Derive its mathematical formulation and explain its intuition.

[ESSENTIAL]

Conversational Answer

Perplexity measures how "surprised" a language model is by real text — it's the exponentiated cross-entropy loss, and intuitively it represents the average number of equally likely word choices the model was weighing at each step. Lower perplexity means the model assigned higher probability to what actually happened.

Intuitive Example

A model with perplexity 3 on a test sentence is, on average, about as uncertain at each step as if it were choosing uniformly among 3 equally likely candidate words — a perplexity of 1 would mean the model predicted every next word with total certainty.

Key Interview Points

- Exponentiated cross-entropy loss
- Intuition: average effective branching factor
- Only comparable across models sharing the same tokenizer/vocabulary

---

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

$$\mathrm{PPL}(W) = P(w_1, \ldots, w_m)^{-\frac{1}{m}} = e^{\mathcal{L}_{\mathrm{CE}}}$$

For a 3-token sequence with $P(w_1) = 0.5$, $P(w_2 \mid w_1) = 0.2$, $P(w_3 \mid w_2) = 0.4$: the joint probability is 0.04, giving cross-entropy $\mathcal{L}_{\mathrm{CE}} = -\frac{1}{3}\ln(0.04) \approx 1.073$ and $\mathrm{PPL} = e^{1.073} \approx 2.924$ — the model was, on average, as uncertain as choosing among roughly 3 candidates at each step.

Production Perspective & Trade-offs

Perplexity depends directly on vocabulary size $|V|$ — a model with a smaller vocabulary has inherently fewer options to choose from at each step and will show a lower perplexity for that reason alone, independent of actual quality. Comparing perplexity across models is only valid when both share the exact same tokenizer and vocabulary. It's also brittle to OOV: a single zero-probability token can send perplexity to infinity, which is why production evaluation pipelines clip or smooth it.

Common Mistakes

1. Comparing perplexity scores across models that use different tokenizers or vocabulary sizes — the comparison is not meaningful.
2. Treating low perplexity as proof of high-quality generation — it only measures next-token prediction confidence, not factual correctness or coherence.

COMMON FOLLOW-UP QUESTIONS

> **Q: How does vocabulary size impact perplexity comparisons?**
>
> A model choosing from a smaller vocabulary at each step has a structurally easier prediction task, so it will tend to show lower perplexity even if it isn't actually a better model.

> **Q: What causes perplexity to spike to infinity?**
>
> A test-time token that the model (or its tokenizer) assigns zero probability to — since perplexity is the reciprocal geometric mean of the sequence probability, a single zero collapses the whole score.

One-Line Takeaway

> **Takeaway:** Perplexity is the exponentiated cross-entropy loss — a useful measure of next-token confidence, but only comparable between models sharing an identical tokenizer and vocabulary.

Question 19: Explain how CBOW and Skip-gram learn word embeddings.

[ESSENTIAL]

Conversational Answer

Both are Word2Vec training objectives that learn embeddings from local context, just in opposite directions. CBOW averages the surrounding context words to predict the center target word. Skip-gram does the reverse — it uses the center word to predict each surrounding context word individually.

Intuitive Example

For the window "the [cat] sat", CBOW averages the embeddings of "the" and "sat" to predict "cat"; Skip-gram instead starts from "cat" and separately tries to predict "the" and "sat" as its context outputs.

Key Interview Points

- CBOW: context → target (averaged input)
- Skip-gram: target → context (multiple predictions per word)

- Both are self-supervised — no labels required

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Both architectures are trained with the same negative-sampling objective; they differ only in which side of the (target, context) pair is the input vs. output.

	CBOW	Skip-gram
Direction	Context → Target	Target → Context
Context handling	Averaged into one vector	Each context word predicted separately
Training speed	Faster (fewer predictions per window)	Slower (one prediction per context word)
Rare word quality	Weaker — averaging dilutes rare signal	Stronger — direct gradient per context word

Production Perspective & Trade-offs

CBOW's averaging step smooths out the contribution of any single context word, which trains faster but weakens the signal for rare words. Skip-gram runs more predictions per training window (slower), but because it doesn't average, rare words get direct, undiluted gradient updates — this is why Skip-gram is generally preferred when rare-word quality matters.

Common Mistakes

1. Assuming Word2Vec requires labeled training data — it's self-supervised, using the surrounding text itself as the label.
2. Assuming CBOW and Skip-gram always produce similar quality embeddings regardless of corpus size — the gap is most visible specifically on rare words.

COMMON FOLLOW-UP QUESTIONS

> **Q: What optimization techniques speed up Word2Vec training for either architecture?**
>
> Negative Sampling and Hierarchical Softmax, both of which avoid the full-vocabulary softmax normalization on every training step.

> **Q: Why does averaging context vectors in CBOW hurt rare-word representations specifically?**
>
> Averaging blends the rare word's contextual signal together with more common neighboring words, diluting exactly the gradient signal that a rare word needs the most to train a good vector from limited occurrences.

One-Line Takeaway

> **Takeaway:** CBOW predicts the target from averaged context (fast, weaker on rare words); Skip-gram predicts context from the target (slower, stronger on rare words) — same objective, opposite direction.

Question 20: Why does Negative Sampling make Word2Vec training faster?

[ESSENTIAL]

Conversational Answer

Standard Softmax over the full vocabulary means every training step has to normalize over potentially millions of words, which is far too slow. Negative sampling turns this into a much cheaper binary classification problem: for each true (target, context) pair, you also sample a handful of random "wrong" words, and the model just has to learn to tell the real pair apart from the fake ones.

Intuitive Example

Instead of asking "which one of these 1,000,000 words comes next?" at every step, negative sampling asks a far cheaper question: "is 'sat' more likely to follow 'cat' than these 5 random noise words like 'refrigerator' or 'purple'?"

Key Interview Points

- Full Softmax bottleneck: normalizes over the entire vocabulary $|V|$
- Binary reformulation: true pair vs. k sampled noise pairs
- Complexity reduction: from $O(|V|)$ to $O(k)$ per update

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

$$\mathcal{L}_{\text{SGNS}} = -\ln \sigma(\mathbf{v}_w \cdot \mathbf{v}'_{w_c}) - \sum_{i=1}^{k} \ln \sigma(-\mathbf{v}_w \cdot \mathbf{v}'_{w_i})$$

The first term pulls the target and true context vector together; the sum pushes the target away from k sampled negative vectors. Negatives are drawn from a smoothed unigram distribution $P_n(w) \propto U(w)^{0.75}$, which boosts the sampling chance of rare words relative to raw frequency so common stopwords don't dominate every negative batch.

Production Perspective & Trade-offs

Only the vectors involved in a given step — the target, the true context, and k negatives — get gradient updates. For $|V| = 10^6$ and $k = 5$, that's a difference of several orders of magnitude in per-step compute versus updating (implicitly, via the softmax normalization) the entire output matrix. This is what makes training embeddings on web-scale corpora tractable on commodity hardware.

Common Mistakes

1. Believing negative sampling updates all vocabulary weights at every step — it only touches the sampled subset.
2. Picking k without considering corpus size: too few negatives on a large, diverse corpus under-constrains rare-word vectors; too many negatives on a small corpus wastes compute for no accuracy gain.

COMMON FOLLOW-UP QUESTIONS

> **Q: What is a reasonable value for k?**
>
> Typically 5-20 for smaller datasets and 2-5 for very large corpora — more negatives help more when there's less data to constrain the embedding space.

> **Q: Why raise the unigram frequency to the power of 0.75 when sampling negatives?**
>
> Raw unigram frequency oversamples extremely common words (like "the"), wasting negative samples on words that provide little contrastive signal; the 0.75 exponent flattens the distribution so rarer words get sampled more often as negatives too.

One-Line Takeaway

> **Takeaway:** Negative sampling replaces an $O(|V|)$ softmax normalization with an $O(k)$ binary classification problem, which is the difference between Word2Vec being trainable on commodity

> hardware or not.

Question 21: Explain mathematically why RNNs suffer from vanishing gradients.

[ESSENTIAL]

Conversational Answer

Backpropagation through time repeatedly multiplies the gradient by the recurrent weight matrix, once per time step. If that matrix tends to shrink vectors (its largest eigenvalue is less than 1), the gradient shrinks exponentially the further back you propagate — by the time it reaches early time steps on a long sequence, there's essentially nothing left to learn from.

Intuitive Example

Multiplying a number by 0.9 repeatedly barely changes it after a few steps but shrinks it to almost nothing after fifty — the gradient signal on a long sequence experiences exactly this decay, step after step, back through time.

Key Interview Points

- Backpropagation Through Time (BPTT)
- Repeated multiplication by the recurrent weight matrix
- Governed by the spectral radius $\rho(W_{hh})$

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

$$\frac{\partial h_T}{\partial h_t} = \prod_{k=t+1}^{T} \mathrm{diag}(1 - h_k^2)\, W_{hh}^T$$

This is a product of $T - t$ matrix terms — if the largest eigenvalue (spectral radius) of W_{hh} is below 1, the product shrinks toward zero as the number of terms grows; if it's above 1, the product blows up instead. Either way, standard RNNs have no mechanism to keep this product near 1 over long ranges.

Production Perspective & Trade-offs

Vanishing gradients are why vanilla RNNs can't learn dependencies spanning more than roughly 10-20 time steps in practice — this single limitation is the direct motivation for LSTM and GRU gating mechanisms, and later for attention-based architectures that bypass the recurrent product entirely.

Common Mistakes

1. Confusing vanishing gradients (a BPTT chain-product effect) with dead ReLU units (a completely different, activation-saturation phenomenon).
2. Assuming a bigger hidden size fixes vanishing gradients — the problem is structural (the recurrent product), not a capacity issue.

COMMON FOLLOW-UP QUESTIONS

> **Q: How does gradient clipping address the *exploding* gradient case?**
>
> If the gradient norm exceeds a threshold, it's rescaled down: $\mathbf{g} \leftarrow \mathbf{g} \cdot \frac{g_{\max}}{\|\mathbf{g}\|}$ — this caps the update size without changing its direction.

> **Q: Does gradient clipping also fix vanishing gradients?**
>
> No — clipping only bounds gradients that are too large; a gradient that has decayed toward zero has nothing left to clip, which is why vanishing requires an architectural fix (gating, residual/additive paths) rather than a training-time patch.

One-Line Takeaway

> **Takeaway:** Vanishing gradients come from repeatedly multiplying by the recurrent weight matrix during BPTT — if its spectral radius is below 1, the gradient signal decays exponentially with sequence length.

Question 22: Explain how LSTM's cell state mitigates the vanishing gradient problem.

[ESSENTIAL]

Conversational Answer

Standard RNNs propagate gradients through repeated matrix multiplication, which is what causes vanishing gradients. LSTMs instead route the gradient through an *additive* cell-state update — as long as the forget gate stays open, the gradient can flow backward across many time steps without being multiplicatively shrunk.

Intuitive Example

Think of the cell state as a conveyor belt running alongside the network rather than through it — information (and gradient) can ride along that belt largely undisturbed, only getting selectively added to or removed from at each station (gate), instead of being reprocessed at every single step.

Key Interview Points

- Additive cell-state update (the "Constant Error Carousel")
- Forget gate controls how much gradient survives each step
- Bypasses the recurrent weight-matrix multiplication entirely

---

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

$$\mathbf{C}_t = \mathbf{f}_t \odot \mathbf{C}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{C}}_t \implies \frac{\partial \mathbf{C}_t}{\partial \mathbf{C}_{t-1}} \approx \mathbf{f}_t$$

Because the cell-state update is additive rather than a matrix multiplication, the gradient path back through time is (approximately) just the forget gate value at each step — if $\mathbf{f}_t \approx 1$, gradients propagate across long ranges without the multiplicative decay that plagues vanilla RNNs. This additive shortcut is called the **Constant Error Carousel (CEC)**.

Production Perspective & Trade-offs

The forget gate's calibration is what actually determines whether this helps: if training drives $\mathbf{f}_t \to 0$, the LSTM discards its own protection and can still vanish, just like a vanilla RNN. The gating mechanism also isn't free — four separate linear layers per cell roughly quadruple parameter count and compute versus a vanilla RNN cell, which is why GRUs (which merge cell and hidden state into one) are often preferred in high-throughput production settings, saving about 25% on parameters.

Common Mistakes

1. Believing LSTMs completely eliminate vanishing gradients — they can still vanish if the forget gate saturates toward 0.
2. Assuming the CEC pathway is the *only* gradient path in an LSTM — the gate-computation terms also contribute, just typically as a smaller effect than the additive cell-state path.

COMMON FOLLOW-UP QUESTIONS

> **Q: What happens if the forget gate is always 0?**
>
> The model discards all historical cell-state information at every step, functionally collapsing to something closer to a stateless feedforward network on each input.

> **Q: Why do GRUs save parameters relative to LSTMs?**
>
> GRUs merge the cell state and hidden state into a single state vector and use only two gates instead of three, cutting roughly a quarter of the per-cell parameter count while retaining most of the gradient-flow benefit.

One-Line Takeaway

> **Takeaway:** LSTMs fight vanishing gradients with an additive cell-state pathway gated by the forget gate — protection that only holds as long as the forget gate stays open.

---

3. Production & Engineering Questions (23-30)

Question 23: What challenges arise when deploying NLP models to production?

[ESSENTIAL]

Conversational Answer

The big four in production are: managing GPU/CPU latency budgets (especially for autoregressive generation), handling vocabulary drift as user language evolves, keeping preprocessing perfectly consistent between training and serving, and controlling deployment cost as traffic scales.

Intuitive Example

A model that scores 95% offline accuracy can still fail in production if serving preprocessing lowercases text differently than training did — the model itself never changed, but its effective input distribution silently shifted.

Key Interview Points

- Latency budgets (especially for generation)
- Vocabulary and data drift
- Model quantization for cost control

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

No single formula governs this — it's a systems problem spanning latency, memory, and data consistency simultaneously, and the failure modes are usually silent (degraded accuracy) rather than loud (crashes).

Production Perspective & Trade-offs

Deploying models is a continuous balancing act between latency and accuracy. Quantization is a common lever to reduce VRAM footprint and cost, but it can degrade accuracy specifically on edge cases and rare inputs that weren't well represented during calibration.

Common Mistakes

1. Assuming research-reported accuracy translates directly to production without a latency-optimization pass.
2. Underestimating how much drift and preprocessing skew — not raw model quality — drive most real-world accuracy degradation.

COMMON FOLLOW-UP QUESTIONS

> **Q: How does model pruning affect execution speed?**
>
> Pruning zeroes out weights to create sparse matrices, which can bypass computation entirely on hardware with sparse-operation support — but on hardware without that support, it may not speed anything up at all.

> **Q: What's the most common production failure mode that offline evaluation misses?**
>
> Training-serving skew from preprocessing inconsistencies — it's invisible in offline evaluation (which reuses the same pipeline) but shows up immediately once a separate serving codepath diverges.

One-Line Takeaway

> **Takeaway:** Production NLP challenges are mostly about consistency and cost under load — latency budgets, drift, and preprocessing parity — not raw model accuracy.

Question 24: How do you handle vocabulary drift in production systems?

[ESSENTIAL]

Conversational Answer

Vocabulary drift happens when live user input introduces words, slang, or symbols the model's vocabulary never saw during training. The standard defenses are byte-fallback tokenizers (never truly fail on unknown input), periodic retraining on fresh data, and monitoring OOV rates as an early warning signal.

Intuitive Example

A model trained in 2019 has never seen "rizz" or "gyat" as tokens — a byte-fallback tokenizer can still decompose them into valid subword or byte sequences rather than collapsing them to a single unhelpful `<unk>`.

Key Interview Points

- Byte-fallback tokenization
- Scheduled retraining loops
- OOV-rate monitoring as an early signal

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

No formula is needed — this is a monitoring-and-retraining loop, not a mathematical computation.

Production Perspective & Trade-offs

Byte-fallback tokenizers decompose unknown words down to raw bytes, guaranteeing there's never a hard OOV failure, at the cost of slightly longer sequences for genuinely novel terms. This buys time until the next scheduled retraining pass on fresh data can properly absorb the new vocabulary into the model's learned representations.

Common Mistakes

1. Relying on manual dictionary additions to patch vocabulary gaps — this doesn't scale in a dynamic, fast-moving environment.
2. Treating byte-fallback as a full fix rather than a stopgap — it prevents crashes, but the model still lacks a well-trained representation for genuinely new terms until retraining catches up.

COMMON FOLLOW-UP QUESTIONS

> **Q: How does vocabulary size impact model serving cost?**
>
> A larger vocabulary increases the size of the embedding and output projection layers directly, which increases GPU VRAM usage and, at the margins, inference latency.

> **Q: What's a practical leading indicator that vocabulary drift is happening before accuracy visibly degrades?**
>
> A rising OOV or byte-fallback rate in production telemetry — it typically climbs well before downstream accuracy metrics show a measurable dip.

One-Line Takeaway

> **Takeaway:** Byte-fallback tokenization prevents vocabulary drift from ever hard-failing; scheduled retraining is what actually closes the representation gap it creates.

Question 25: What is training-serving skew? Why is it problematic?

[ESSENTIAL]

Conversational Answer

Training-serving skew is when the preprocessing pipeline, feature computation, or data distribution differs between training time and serving time — even subtly — causing the deployed model to see inputs it was never actually trained on.

Intuitive Example

If training lowercases text with Python's `.lower()` but the serving layer (written in a different language, say C++) uses a slightly different Unicode-aware lowercasing routine, the same input string can tokenize differently in each environment — and the model silently sees a distribution it never trained on.

Key Interview Points

- Preprocessing consistency between training and serving
- Feature computation drift
- Pipeline version alignment

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

No formula applies — this is a systems-consistency problem, not a computation.

Production Perspective & Trade-offs

The standard fix is to wrap tokenization and preprocessing into a single, version-controlled shared library used identically by both the training pipeline and the serving layer, eliminating the possibility of two implementations drifting apart over time.

Common Mistakes

1. Maintaining separate training (e.g. Python) and serving (e.g. C++) implementations without exhaustive tests that assert identical outputs on the same inputs.
2. Assuming a preprocessing change is "safe" without re-validating it against both the training and serving code paths.

COMMON FOLLOW-UP QUESTIONS

> **Q: How does Unicode decomposition specifically cause training-serving skew?**
>
> If the serving tokenizer normalizes Unicode differently than training did (e.g. NFD vs. NFC), the same visual text can split into different token sequences, mapping to different — and wrong — vocabulary indices.

> **Q: What's the most reliable way to prevent training-serving skew long-term?**
>
> Share one preprocessing implementation across both training and serving (not just similar logic in two languages), so there is structurally no way for the two to diverge.

One-Line Takeaway

> **Takeaway:** Training-serving skew is a silent accuracy killer caused by any divergence between training-time and serving-time preprocessing — the fix is one shared, versioned pipeline, not two parallel ones.

Question 26: Explain Data Drift versus Concept Drift with examples.

[ESSENTIAL]

Conversational Answer

Data drift is when the input distribution shifts but the relationship between inputs and labels stays the same — new vocabulary, same meaning of "spam." Concept drift is when that input-label relationship itself changes — the same word or pattern now means something different than it used to.

Intuitive Example

A sentiment classifier suddenly seeing lots of TikTok slang is data drift (new inputs, same task). The word "viral" shifting from a negative health-quarantine connotation in 2019 to a positive marketing connotation in 2021 is concept drift (the label meaning itself moved).

Key Interview Points

- Data (covariate) drift: $P(X)$ changes, $P(Y|X)$ stable
- Concept drift: $P(Y|X)$ itself changes
- Different monitoring strategies for each

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Data drift is a shift in $P(X)$ with $P(Y|X)$ held fixed; concept drift is a shift in $P(Y|X)$ itself — the practical distinction is *what* changed, and it determines whether monitoring input distributions alone is sufficient or whether you need labeled feedback.

Production Perspective & Trade-offs

Data drift can be caught by monitoring input feature distributions alone (e.g. PSI on vocabulary bins), with no labels required. Concept drift is fundamentally harder to detect this way, since the inputs may look completely unchanged — it requires tracking labeled performance metrics over time, which usually means a slower feedback loop.

Common Mistakes

1. Retraining on raw new inputs to "fix" concept drift without first verifying the label mappings themselves are still correct.
2. Assuming input-distribution monitoring (PSI, KS test) is sufficient to catch concept drift — it isn't, since concept drift can occur with no visible change in $P(X)$.

COMMON FOLLOW-UP QUESTIONS

> **Q: How do you monitor for concept drift specifically?**
>
> By tracking labeled performance metrics (F1, accuracy) on a stream of production data over time — since concept drift doesn't necessarily show up in the input distribution alone.

> **Q: Which is generally easier to detect automatically, data drift or concept drift?**
>
> Data drift — it can be flagged from unlabeled input distributions alone (e.g. PSI), whereas concept drift requires ongoing labeled feedback, which is slower and more expensive to collect.

One-Line Takeaway

> **Takeaway:** Data drift changes what inputs look like; concept drift changes what they mean — and only the first is detectable from unlabeled data alone.

Question 27: When would you choose batch inference over real-time inference?

[ESSENTIAL]

Conversational Answer

Choose batch inference when throughput matters more than immediate response — like running a classification sweep over last night's logs. Choose real-time inference when you're serving interactive user-facing requests with strict latency limits, like search auto-completion.

Intuitive Example

Scoring 10 million historical support tickets for topic classification overnight is a batch job — nobody's waiting on any single result. Auto-completing a user's search query as they type has a hard sub-100ms budget per keystroke — that has to be real-time.

Key Interview Points

- High throughput (batch) vs. low latency (real-time)
- GPU utilization profile differs sharply between the two
- Cost per token/request is generally lower for batch

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

No formula is required — the decision is governed entirely by the latency SLA of the use case, not by any computed quantity.

Production Perspective & Trade-offs

Batch inference maximizes GPU utilization by processing large chunks of data together, which lowers cost per unit of work. Real-time inference processes requests as they arrive, often leaving the GPU underutilized between requests unless dynamic batching is used to opportunistically group concurrent requests.

Common Mistakes

1. Deploying an always-on real-time server for a workload that could run cheaper as a scheduled batch job.
2. Ignoring dynamic batching as a middle-ground option when real-time latency requirements aren't extremely tight.

COMMON FOLLOW-UP QUESTIONS

> **Q: How does dynamic batching optimize real-time inference?**
>
> It groups multiple concurrent incoming requests into a single batch at the server level within a short time window, trading a small amount of added latency for significantly higher GPU throughput.

> **Q: What's a good heuristic for choosing batch vs. real-time?**
>
> If a human or downstream system is waiting synchronously on the result, it needs real-time serving; if the results are consumed later (dashboards, reports, retraining data), batch is almost always cheaper.

One-Line Takeaway

> **Takeaway:** Batch inference optimizes for throughput and cost; real-time inference optimizes for latency — the workload's actual SLA, not preference, should decide.

Question 28: How do quantization and pruning reduce inference cost?

[ESSENTIAL]

Conversational Answer

Quantization converts model weights from 32-bit floats down to lower-precision formats like float16 or int8, directly shrinking VRAM footprint and memory-bandwidth cost. Pruning zeroes out less-important weights to create sparse matrices, which can skip computation entirely on hardware that supports sparse operations.

Intuitive Example

Converting a 7-billion-parameter model from float32 to int8 roughly quarters its memory footprint — the difference between needing an expensive multi-GPU server and fitting comfortably on a single consumer GPU.

Key Interview Points

- Quantization: precision conversion (fp32 → fp16/int8)

- Pruning: weight sparsity
- Both target VRAM footprint, via different mechanisms

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

No formula is required — the mechanisms are precision reduction (quantization) and structured/unstructured zeroing (pruning), and their actual speed benefit depends on hardware support, not just the technique itself.

	Quantization	Pruning
Mechanism	Lower numeric precision (fp32 → fp16/int8)	Zero out less-important weights
VRAM savings	Up to ~75% (fp32 → int8)	Depends on sparsity level achieved
Speed gain	Consistent across most hardware	Requires hardware sparse-op support to realize
Accuracy risk	Can degrade on edge cases without QAT	Aggressive pruning can silently drop capacity

Production Perspective & Trade-offs

Quantization can reduce VRAM usage by up to roughly 75% (fp32 → int8), enabling deployment on cheaper hardware, but it risks degrading accuracy specifically on edge cases unless done carefully. Pruning's speed benefit is conditional — it only pays off on hardware that can actually exploit sparse matrix operations; on hardware without that support, a pruned model may run no faster at all.

Common Mistakes

1. Assuming pruning always speeds up inference — it only helps on hardware with sparse-operation support.
2. Applying aggressive post-training quantization without evaluating edge-case accuracy specifically, not just aggregate metrics.

COMMON FOLLOW-UP QUESTIONS

> **Q: What is Post-Training Quantization (PTQ) vs. Quantization-Aware Training (QAT)?**
>
> PTQ quantizes an already-trained model's weights after the fact — fast but can lose more accuracy; QAT simulates quantization *during* training so the model learns to be robust to the precision loss, generally yielding better final accuracy.

> **Q: Why doesn't pruning always translate to a real speedup?**
>
> Zeroing weights only saves compute if the underlying hardware and kernels can skip multiplying by zero — without sparse-operation support, the hardware still performs the same dense matrix multiplication regardless of how many weights are zero.

One-Line Takeaway

> **Takeaway:** Quantization reliably shrinks VRAM footprint across most hardware; pruning's speedup is conditional on sparse-operation hardware support — neither is free of accuracy risk.

Question 29: What preprocessing inconsistencies can lead to degraded model performance?

[ESSENTIAL]

Conversational Answer

The usual suspects are: different casing rules, different regex cleanup logic, different Unicode normalization forms (NFC vs. NFD), or a subword vocabulary mismatch between training and serving. Any one of these silently shifts the effective input distribution the model sees at inference time.

Intuitive Example

If training strips emoji but the serving pipeline doesn't, a sentiment classifier suddenly sees tokens it never trained on every time a user includes an emoji — a subtle, hard-to-spot production regression.

Key Interview Points

- Token normalization mismatch
- Unicode normalization form mismatch (NFC/NFD)
- Requires explicit pipeline parity testing

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

No formula applies — the failure mode is a discrete pipeline mismatch, not a continuous computed quantity.

Production Perspective & Trade-offs

A single missed preprocessing step — like inconsistent punctuation cleaning — can cause the tokenizer to split words differently between training and serving, silently mapping them to the wrong vocabulary indices. Complex regex cleanup (especially with lookaheads) can itself become a CPU latency bottleneck in high-throughput serving paths, separate from the correctness issue.

Common Mistakes

1. Assuming standard tokenizer libraries handle raw, uncleaned text consistently across environments without explicit normalization.
2. Not writing parity tests that assert training and serving preprocessing produce byte-identical output on the same input.

COMMON FOLLOW-UP QUESTIONS

> **Q: How does regex-based cleaning impact serving latency?**
>
> Complex regex patterns, especially those using lookaheads/lookbehinds, run on CPU and can become a measurable bottleneck in high-throughput, low-latency serving pipelines.

> **Q: What's the most reliable way to catch preprocessing inconsistencies before they reach production?**
>
> Automated parity tests that run the same raw inputs through both the training and serving preprocessing code paths and assert identical token-ID output.

One-Line Takeaway

> **Takeaway:** Preprocessing inconsistencies are invisible until they hit production — the fix is automated parity testing between training and serving pipelines, not code review alone.

Question 30: What monitoring metrics would you track for an NLP model in production?

[ESSENTIAL]

Conversational Answer

Three categories: systems metrics (inference latency, GPU/VRAM usage, error rates), data metrics (OOV rate, input length distribution, drift scores), and performance metrics (prediction confidence distributions, feedback-based accuracy signals).

Intuitive Example

A model can look perfectly healthy on systems metrics (low latency, no errors) while quietly degrading in quality — that's exactly why OOV rate and confidence-distribution monitoring exist as a second, independent layer of signal.

Key Interview Points

- Systems health: latency, GPU/VRAM usage
- Data health: OOV rate, drift scores
- Model health: confidence distributions, feedback accuracy

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

No formula is required — this is a monitoring taxonomy, not a computation, though the drift scores it feeds into (e.g. PSI) do have their own formulas covered elsewhere in this module.

Production Perspective & Trade-offs

Automated alarms on OOV rate are a particularly high-leverage signal: a spike is one of the earliest, cheapest-to-compute indicators of data drift, well before labeled accuracy metrics would show a measurable decline.

Common Mistakes

1. Monitoring only systems metrics (CPU/GPU load, uptime) while neglecting data drift and model-quality signals entirely.
2. Waiting for labeled accuracy metrics to show decline before investigating — by then, the issue has usually been live for a while.

COMMON FOLLOW-UP QUESTIONS

> **Q: How do you measure drift specifically on text embeddings?**
>
> By tracking the distribution of cosine distances between live production embeddings and a fixed reference set of training embeddings over time, watching for a shift in that distribution.

> **Q: Why track prediction confidence distributions as a standalone signal?**
>
> A sudden shift toward lower average confidence often precedes a visible accuracy drop, giving an earlier warning than waiting for labeled feedback to accumulate.

One-Line Takeaway

> **Takeaway:** Production NLP monitoring needs three independent layers — systems, data, and model-quality signals — because systems health alone can look fine while quality quietly degrades.

---

4. Debugging & Evaluation Questions (31-35)

Question 31: Why can BLEU and ROUGE give misleading evaluation results?

[ESSENTIAL]

Conversational Answer

BLEU and ROUGE only measure exact n-gram overlap between candidate and reference text. That makes them blind to synonyms — "feline" vs. "cat" scores as a complete mismatch — and they can even score a grammatically fluent but logically opposite generation (e.g. a flipped negation) surprisingly well.

Intuitive Example

"The movie was not good" and "The movie was good" overlap on 4 of 5 words — BLEU would score them as highly similar, even though they mean the exact opposite.

Key Interview Points

- Exact n-gram overlap only
- Blind to synonyms
- Can score negations/opposites deceptively well

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Both metrics are precision/recall computations over exact n-gram matches — there's no semantic representation involved anywhere in the computation, which is exactly why synonym and negation blind spots exist structurally, not as edge-case bugs.

Production Perspective & Trade-offs

Relying only on BLEU/ROUGE risks shipping models that are fluent and n-gram-similar to references but factually or logically wrong. Production evaluation pipelines pair them with semantic metrics (BERTScore) and periodic human validation loops to catch what overlap metrics structurally cannot.

Common Mistakes

1. Believing a high BLEU score guarantees factual or semantic correctness.
2. Using BLEU/ROUGE as the sole automated gate for shipping a generation model without any semantic or human-in-the-loop check.

COMMON FOLLOW-UP QUESTIONS

> **Q: How does METEOR improve on BLEU?**
>
> METEOR incorporates stemming and synonym matching (via a resource like WordNet) into its alignment step, partially closing the synonym blind spot that plain n-gram overlap has.

> **Q: Why is negation such a dangerous specific failure case for these metrics?**
>
> Flipping a single word like "not" changes only one token out of many, so n-gram overlap barely drops — the metric doesn't understand that this single-token change inverted the entire meaning.

One-Line Takeaway

> **Takeaway:** BLEU and ROUGE measure textual overlap, not meaning — they can be fooled by both synonyms and outright negations, so they need semantic or human backup in production.

Question 32: When would you prefer BERTScore over BLEU?

[ESSENTIAL]

Conversational Answer

Prefer BERTScore whenever synonyms or paraphrasing are acceptable — it computes similarity in embedding space, so "feline" and "cat" score as close even though they share no characters. Prefer

BLEU/ROUGE when you need fast, cheap, exact-match regression testing, like tracking a translation system's output against fixed references over time.

Intuitive Example

Candidate "A feline rested on the rug" against reference "The cat sat on the mat" shares almost no exact words, so BLEU would score it harshly — but BERTScore recognizes "feline"≈"cat" and "rug"≈"mat" as high-similarity pairs in embedding space and scores it much more fairly.

Key Interview Points

- BERTScore: embedding-space greedy alignment
- BLEU/ROUGE: exact n-gram overlap
- Trade-off is semantic accuracy vs. evaluation latency/cost

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

$$F_{\text{BERT}} = 2 \times \frac{P_{\text{BERT}} \times R_{\text{BERT}}}{P_{\text{BERT}} + R_{\text{BERT}}}$$

where R_{BERT} and P_{BERT} are computed by greedily matching each reference (or candidate) token to its highest-cosine-similarity counterpart in the other sequence, using contextual embeddings — high pairwise similarity scores (e.g. "feline"↔"cat" at 0.88) drive the overall F1 up even with zero exact-string overlap.

Production Perspective & Trade-offs

BLEU is a pure string-overlap check running in well under a millisecond on CPU. BERTScore requires a full transformer forward pass to extract contextual embeddings for every token, which is orders of magnitude slower and GPU-bound — making it expensive to run over large evaluation sets in a tight CI loop. BERTScore's quality is also only as good as its underlying embedding model, so the encoder choice (e.g. RoBERTa-large) matters and can introduce its own bias.

Common Mistakes

1. Ignoring BERTScore's compute cost when evaluating very large datasets, where it can become the evaluation pipeline's bottleneck.
2. Assuming BERTScore is bias-free — its scores inherit whatever biases exist in the underlying pretrained embedding model.

COMMON FOLLOW-UP QUESTIONS

> **Q: How does BERTScore compute its greedy alignment?**
>
> It computes pairwise cosine similarities between every candidate token embedding and every reference token embedding, then matches each token to its single highest-similarity counterpart in the other sequence.

> **Q: Why not just always use BERTScore instead of BLEU?**
>
> Cost and latency — BERTScore needs a transformer forward pass per evaluation, making it impractical for very large-scale or CI-speed regression testing where BLEU's near-instant string comparison is the better fit.

One-Line Takeaway

> **Takeaway:** BERTScore catches semantic equivalence that BLEU structurally cannot, at the cost of needing a transformer forward pass per comparison — pick based on whether you need semantic accuracy or evaluation speed.

Question 33: How would you debug an NLP classifier with poor precision but high recall?

[ESSENTIAL]

Conversational Answer

High recall with poor precision means the model is over-predicting the positive class — catching everything, but with too many false positives along the way. The fix path is: raise the decision threshold, inspect the confusion matrix for patterns in the false positives, and check for class imbalance in training data.

Intuitive Example

A spam filter that flags every promotional email as spam has high recall (it never misses real spam) but low precision (it also blocks valid marketing emails users wanted) — annoying, but a different failure mode than missing spam entirely.

Key Interview Points

- High false positive rate
- Decision-threshold tuning
- Class imbalance as a root cause to check first

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \qquad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

Raising the classification threshold directly trades recall for precision by requiring higher confidence before predicting positive — it's the fastest lever to pull before considering retraining.

Production Perspective & Trade-offs

In spam filtering specifically, high recall with low precision means the model over-blocks legitimate email — the practical fix is almost always to raise the threshold first, since it requires no retraining and can be deployed immediately, before investigating deeper data issues.

Common Mistakes

1. Jumping straight to retraining from scratch before trying the much cheaper threshold adjustment first.
2. Not checking the confusion matrix for a pattern in false positives (e.g. one specific category driving most of the error) before assuming it's a general model-quality issue.

COMMON FOLLOW-UP QUESTIONS

> **Q: How does the F1 score help evaluate this precision/recall trade-off?**
>
> F1 is the harmonic mean of precision and recall, so it penalizes threshold choices that push either one to an extreme — it's a useful single number for finding a balanced operating point.

> **Q: When is high recall worth accepting low precision, deliberately?**
>
> When false negatives are far more costly than false positives — e.g. flagging potential fraud for human review, where missing real fraud is worse than a reviewer occasionally checking a false alarm.

One-Line Takeaway

> **Takeaway:** High recall with low precision means the model over-predicts positive — try threshold tuning first, since it's immediate and free, before touching the model itself.

Question 34: How would you diagnose exploding gradients during RNN training?

[ESSENTIAL]

Conversational Answer

Exploding gradients show up as training loss suddenly spiking to NaN, weight values diverging toward infinity, or unusually large gradient norms during backpropagation. The standard fixes are gradient clipping and reducing the learning rate.

Intuitive Example

A training run with a smoothly decreasing loss curve that suddenly jumps to `NaN` at step 4,200, with no other data changes, is the classic exploding-gradient signature — worth checking gradient norm logs at that exact step before suspecting a data issue.

Key Interview Points

- Loss spikes to NaN
- Gradient norm monitoring
- Gradient clipping as the standard fix

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

$$\mathbf{g} \leftarrow \mathbf{g} \cdot \frac{g_{\max}}{||\mathbf{g}||} \quad \text{if } ||\mathbf{g}|| > g_{\max}$$

Gradient clipping rescales the gradient vector to a maximum norm while preserving its direction, capping how large a single update step can be without changing what direction it points.

Production Perspective & Trade-offs

Gradient clipping is a targeted fix for exploding gradients specifically — it does nothing for vanishing gradients, which require an architectural fix like LSTM/GRU gating instead. A max-norm threshold of 1.0 is a common, reasonable default starting point.

Common Mistakes

1. Assuming `NaN` loss always comes from a data bug (e.g. division by zero in preprocessing) rather than checking gradient norms first.

2. Applying gradient clipping and assuming it also addresses vanishing gradients — it's a fix for the opposite failure mode.

COMMON FOLLOW-UP QUESTIONS

> **Q: What max-norm value is typically used for clipping?**
>
> A threshold of 1.0 is a standard, widely used default in deep learning training pipelines, though it's sometimes tuned per architecture.

> **Q: Besides clipping, what else helps prevent exploding gradients?**
>
> Lowering the learning rate, using a more stable optimizer, or careful weight initialization all reduce the chance of the recurrent weight product growing unboundedly large in the first place.

One-Line Takeaway

> **Takeaway:** A loss spike to NaN with large gradient norms is the exploding-gradient signature — gradient clipping is the direct fix, but it does nothing for vanishing gradients.

Question 35: How would you perform error analysis for an NLP system?

[ESSENTIAL]

Conversational Answer

Log every prediction, filter down to cases where the prediction disagrees with ground truth, manually read through a representative sample (100-200 errors is a good starting size), bucket the errors into categories (tokenization bugs, typos, class bias, etc.), and target fixes at whichever category is most common.

Intuitive Example

Reading through 150 misclassified support tickets might reveal that 60% of errors come from tickets containing product names the tokenizer splits oddly — a specific, fixable pattern that aggregate accuracy alone would never surface.

Key Interview Points

- Manual review of a representative error sample
- Bucketing errors into root-cause categories
- Targeted fixes over blind retraining

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

No formula is required — this is a qualitative auditing process, though its output (error category frequencies) is often summarized with the same precision/recall/F1 metrics used elsewhere.

Production Perspective & Trade-offs

Systematic error analysis reveals specific, fixable preprocessing bugs or drift patterns that aggregate metrics hide entirely — this lets you ship a targeted fix (e.g. a tokenization rule change) instead of an expensive, unfocused full retrain.

Common Mistakes

1. Relying only on aggregate metrics (accuracy, F1) without ever auditing individual error logs by hand.
2. Reviewing too small or non-representative an error sample to reliably identify a dominant failure category.

COMMON FOLLOW-UP QUESTIONS

> **Q: How does class imbalance complicate error analysis?**
>
> It tends to inflate false negatives for the minority class specifically, since the model has learned to favor predicting the majority class — error analysis needs to account for this skew rather than reading raw error counts at face value.

> **Q: How large should a manual error sample be to draw reliable conclusions?**
>
> 100-200 errors is a common practical starting point — large enough to spot recurring patterns, small enough to review by hand in a reasonable amount of time.

One-Line Takeaway

> **Takeaway:** Manual, categorized error analysis surfaces specific fixable bugs that aggregate metrics hide — it's the difference between a targeted patch and a blind retrain.

---

5. System Design & Applied Questions (36-40)

Question 36: Design a spam email classifier for millions of users.

[ESSENTIAL]

Conversational Answer

I'd build a multi-tier pipeline: normalize the raw email and HTML, apply Feature Hashing to keep a zero-growth vocabulary footprint at scale, run a cheap first-pass classifier (logistic regression or Naive Bayes) on everything, and route only low-confidence cases to a heavier secondary model like an LSTM or transformer.

Intuitive Example

99% of spam is obvious — "WIN A FREE PRIZE NOW" doesn't need a transformer to catch. Reserving the expensive model only for the ambiguous middle ground keeps average latency and cost low while still catching the hard cases.

Key Interview Points

- High-throughput preprocessing pipeline
- Feature Hashing trick for bounded vocabulary memory
- Multi-tier classification: cheap filter first, expensive model on low-confidence cases

---

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Feature hashing maps arbitrary tokens into a fixed-size hash bucket space, so the feature vector dimensionality never grows with vocabulary size — this is what keeps the first-tier model's memory footprint constant regardless of how much new vocabulary appears over time.

Production Perspective & Trade-offs

The tiered design directly targets both scale and latency: Feature Hashing avoids ever needing to store or grow a full vocabulary dictionary, and routing only ambiguous emails to the expensive second-tier model keeps average serving cost low while still catching hard cases. Monitoring OOV/hash-collision rates and user spam reports feeds back into detecting vocabulary drift over time.

Common Mistakes

1. Proposing a heavy transformer model for the full email volume, which is unnecessary cost and latency for the ~99% of cases a cheap model already handles correctly.

2. Forgetting to design a feedback loop (user reports, low-confidence review) that catches drift in spam patterns over time.

COMMON FOLLOW-UP QUESTIONS

> **Q: How do you handle attachments in this pipeline?**
>
> Extract text from attachments with parser libraries and feed it through the same classification pipeline, or route attachments to a separate dedicated file-scanning pipeline depending on risk tolerance.

> **Q: Why is Feature Hashing preferred over a standard vocabulary dictionary at this scale?**
>
> A standard dictionary grows unboundedly as new vocabulary appears; feature hashing maps everything into a fixed-size space up front, trading a small, tunable rate of hash collisions for a memory footprint that never grows.

One-Line Takeaway

> **Takeaway:** A tiered spam classifier — cheap first-pass filter plus expensive model only on ambiguous cases — is what makes millions-of-users scale affordable without sacrificing accuracy on hard cases.

Question 37: Design an autocomplete/search suggestion system.

[ESSENTIAL]

Conversational Answer

I'd use a trie populated with query frequencies. For a given prefix, look up the top-K completions ranked by MLE probability, and apply smoothing (Katz backoff or interpolation) so rare or unseen prefixes still return reasonable suggestions instead of nothing.

Intuitive Example

Typing "how to fi" should instantly surface "how to fix a leak," "how to file taxes," etc. — a trie lets you walk directly to the "fi" node and read off its highest-frequency children in constant time relative to the prefix length, no full-corpus scan needed.

Key Interview Points

- Trie structure for prefix lookups
- MLE-based ranking of completions
- Smoothing for rare/unseen prefixes

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

A trie lookup for a prefix of length k costs $O(k)$ regardless of how many total queries are indexed, which is what makes sub-10ms suggestion latency achievable even over a massive query log.

Production Perspective & Trade-offs

Suggestion latency needs to stay under roughly 10ms, so the trie is typically kept fully in-memory (e.g. via Redis) with frequent prefixes cached. Storage is controlled by pruning low-frequency n-grams (e.g. count < 5) out of the trie entirely, trading a small amount of suggestion coverage for a much smaller memory footprint.

Common Mistakes

1. Querying a SQL database directly per keystroke for suggestions — far too slow for real-time interactive latency budgets.
2. Not pruning low-frequency entries, letting the trie grow unboundedly with rarely-useful long-tail queries.

COMMON FOLLOW-UP QUESTIONS

> **Q: How do you personalize suggestions per user?**
>
> Re-weight the trie's completion probabilities using the individual user's historical search categories or past queries, blended with the global frequency ranking.

> **Q: Why does Katz backoff matter here specifically?**
>
> A brand-new or rare prefix may have very few or zero observed completions; backoff falls back to a shorter, better-populated prefix's statistics rather than returning nothing or a poorly-estimated result.

One-Line Takeaway

> **Takeaway:** An in-memory trie keyed by prefix, ranked by smoothed query frequency, is what makes sub-10ms autocomplete possible at scale.

Question 38: Design a sentiment analysis pipeline for social media.

[ESSENTIAL]

Conversational Answer

I'd keep emojis and punctuation during preprocessing (they're strong sentiment signals, not noise), tokenize with a subword method like BPE to handle heavy slang and typos, run a bidirectional classifier for full-sentence context, and monitor for data drift continuously since social media vocabulary shifts fast.

Intuitive Example

Stripping "😭😭😭" or "!!!" during cleanup would throw away some of the strongest sentiment signal in a tweet — unlike formal text, punctuation and emoji density are themselves informative features here, not noise to remove.

Key Interview Points

- Preserve emojis/punctuation as sentiment signal
- Subword tokenization for slang and typo robustness
- Bidirectional context + continuous drift monitoring

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

No formula is required — the design choices (what to preprocess out, which architecture) follow directly from the domain's characteristics: high informal-language variance and fast-moving vocabulary.

Production Perspective & Trade-offs

Social media text carries unusually high rates of typos, slang, and emoji use — subword tokenization and preserved punctuation are what let the model extract signal from exactly that noise instead of discarding it. Because vocabulary shifts quickly on social platforms, drift monitoring (e.g. PSI on token distributions) needs a tighter check interval than it would for a more static text domain.

Common Mistakes

1. Discarding emojis and punctuation during preprocessing as if they were noise, removing genuinely valuable sentiment signal.
2. Under-provisioning drift monitoring frequency for a domain that changes vocabulary faster than most.

COMMON FOLLOW-UP QUESTIONS

> **Q: Why specifically use a bidirectional model here?**
>
> Negation words like "not" can appear before or after the sentiment-bearing word ("not good" vs. "good, not really"), and bidirectional context lets the model resolve either ordering correctly.

> **Q: How would you validate that emoji/punctuation preservation is actually helping?**
>
> Run an ablation — train and evaluate an otherwise-identical model with those signals stripped, and compare accuracy on a held-out social media test set.

One-Line Takeaway

> **Takeaway:** Social media sentiment analysis works best when preprocessing treats emojis and punctuation as signal, not noise, and drift monitoring runs on a tighter cadence than a typical NLP pipeline.

Question 39: Design an FAQ chatbot using classical NLP techniques (without LLMs).

[ESSENTIAL]

Conversational Answer

Preprocess the incoming query, compute its TF-IDF representation, rank the FAQ database with BM25, and return the answer tied to the highest-scoring match. It's fast, deterministic, and needs no GPU — the trade-off is that it can't handle paraphrased queries that don't share vocabulary with the stored questions.

Intuitive Example

A user asking "how do I get my money back" won't match a stored FAQ titled "Refund Policy" via keyword overlap alone — this is exactly the gap that motivates adding a semantic layer on top of the keyword baseline.

Key Interview Points

- BM25 keyword matching against an FAQ index
- Fast, CPU-only, deterministic
- Blind to paraphrasing/synonyms without a semantic layer

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

This reuses the same BM25 ranking function covered in Module 04 (Vector Space Models) — the FAQ system is effectively a small-scale search-and-retrieve problem, with each FAQ entry treated as a "document" to be ranked against the incoming query.

Production Perspective & Trade-offs

The pure keyword-matching approach is extremely cheap to run (CPU-only, no model inference) and fully deterministic, which makes debugging easy — but it fundamentally cannot bridge a vocabulary gap between how a user phrases a question and how the FAQ is worded. Adding Sentence-BERT embeddings alongside BM25 (a hybrid search) closes that gap while keeping BM25 as a fast, reliable fallback.

Common Mistakes

1. Proposing a full generative model for what is fundamentally a retrieval problem over a small, fixed answer set.
2. Not handling spelling errors, which can break exact keyword matching entirely on an otherwise-correct query.

COMMON FOLLOW-UP QUESTIONS

> **Q: How do you handle spelling errors in the incoming query?**
>
> Apply a spelling-correction pass before matching, or use subword-based similarity (FastText-style n-grams) so a misspelled query still overlaps meaningfully with the correct FAQ entry.

> **Q: What's the minimal upgrade path from this pure-BM25 system to something more capable?**
>
> Add a semantic embedding layer (e.g. Sentence-BERT) as a second ranking signal alongside BM25, forming a hybrid retriever that catches paraphrases the keyword-only approach misses.

One-Line Takeaway

> **Takeaway:** A BM25-ranked FAQ matcher is fast, cheap, and deterministic, but it can't bridge a vocabulary gap — that's exactly what a hybrid semantic layer is for.

Question 40: Given an NLP application, how would you decide between: TF-IDF, Word2Vec, FastText, LSTM, Transformer?

[ESSENTIAL]

Conversational Answer

I'd start from the cheapest model that could plausibly work and only upgrade if the accuracy gain justifies the added latency and cost — TF-IDF for simple keyword-driven tasks, Word2Vec for static semantic similarity, FastText when typos or OOV are a real concern, LSTM for moderate-length sequences with limited compute, and Transformers when accuracy is the priority and you have the compute budget to support them.

Intuitive Example

A basic FAQ matcher probably only needs TF-IDF or BM25; a chatbot handling open-ended, nuanced user questions across long contexts almost certainly needs a Transformer — the right choice tracks task complexity and latency budget, not "what's newest."

Key Interview Points

- Latency vs. accuracy trade-off
- OOV handling needs
- Available compute/hosting resources

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

No single formula governs the choice — it's a decision matrix over task complexity, latency budget, OOV sensitivity, and available compute.

Model	Best for	Key limitation
TF-IDF	Simple keyword search/classification, tight compute budgets	No semantic similarity
Word2Vec	Static semantic similarity lookups	No context-sensitivity (polysemy)
FastText	OOV-heavy, typo-prone, or morphologically rich text	Larger memory footprint than Word2Vec

Model	Best for	Key limitation
LSTM	Moderate-length sequences, limited training resources	Sequential — can't parallelize training
Transformer	Maximum accuracy, long context, ample compute	Highest latency/cost, quadratic attention memory

Production Perspective & Trade-offs

The standard production pattern is to start with a cheap baseline (TF-IDF or Word2Vec), measure its accuracy ceiling, and only invest in an LSTM or Transformer if the accuracy gain is large enough to justify the added latency and hosting cost — not simply because the newer architecture exists.

Common Mistakes

1. Defaulting straight to a Transformer without first establishing and measuring a simpler baseline.
2. Choosing based on architecture recency rather than the specific task's latency budget and OOV/context requirements.

COMMON FOLLOW-UP QUESTIONS

Q: Which model handles multilingual text best?

FastText or Transformers — both decompose text into subwords/n-grams, which keeps multilingual vocabulary tables manageable compared to a per-language word-level vocabulary.

Q: How do you decide when the accuracy gain from upgrading justifies the cost?

Measure the baseline's accuracy on a held-out set, estimate the serving cost delta of the upgrade, and treat it as a genuine cost-benefit decision tied to the product's actual latency and budget constraints — not a default upgrade path.

One-Line Takeaway

> **Takeaway:** Model selection should start from the cheapest viable baseline and upgrade only when the accuracy gain measurably justifies the added latency and cost.

6. Advanced Questions (41-50)

Question 41: Why is BM25 generally better than TF-IDF for retrieval?

[ESSENTIAL]

Conversational Answer

BM25 improves on TF-IDF in two specific ways: it saturates term-frequency contribution instead of scaling it linearly, so a term repeated 100 times can't dominate a score indefinitely, and it explicitly normalizes for document length so long documents don't win purely by containing more words.

Intuitive Example

A document that repeats "shoes" 50 times shouldn't score 50x higher than one that mentions it twice and is otherwise perfectly relevant — TF-IDF would let it, BM25's saturation curve caps that runaway effect.

Key Interview Points

- Term-frequency saturation via k_1
- Document-length normalization via b
- Sub-linear scaling vs. TF-IDF's linear scaling

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

BM25's term-frequency component saturates toward an asymptote of $(k_1 + 1)$ as raw frequency grows, unlike TF-IDF's unbounded linear scaling; the b parameter additionally scales the score by document length relative to the corpus average, penalizing documents that are simply longer rather than more relevant. Both are covered with full worked calculations in Module 04.

Production Perspective & Trade-offs

BM25 is the default ranking function in production search engines like Elasticsearch precisely because it prevents both of TF-IDF's failure modes — term-frequency dominance and length bias — without requiring any model inference, keeping it fast and CPU-only.

Common Mistakes

1. Believing BM25 requires a neural network or embedding model — it's a pure keyword-count algorithm.

2. Tuning k_1 and b without validating against actual query relevance data, treating the defaults as universal.

COMMON FOLLOW-UP QUESTIONS

> **Q: What's the effect of setting $b = 0$?**
>
> Document-length normalization is fully disabled — document length no longer affects the score at all, which can let long documents dominate again.

> **Q: When would you tune k_1 higher vs. lower?**
>
> A higher k_1 lets term frequency contribute more before saturating (rewarding repeated terms more), while a lower k_1 saturates faster, useful when keyword stuffing is a known problem in the corpus.

One-Line Takeaway

> **Takeaway:** BM25 beats TF-IDF by capping term-frequency dominance and correcting for document length — both without needing any model inference.

Question 42: Why does FastText outperform Word2Vec for rare words?

[ESSENTIAL]

Conversational Answer

Word2Vec learns one independent vector per word, so rare words — by definition seen only a handful of times — end up with poorly trained embeddings. FastText represents every word as a sum of character n-grams, so a rare word can inherit a reasonable vector from subword pieces shared with more common words, even with very few direct occurrences.

Intuitive Example

"biodegradability" might appear only twice in a training corpus — too few for Word2Vec to learn a good vector. FastText instead builds its vector from n-gram pieces like "bio", "degrade", "ability" that appear frequently across many other words, giving it a far better-trained representation despite its own rarity.

Key Interview Points

- Character n-gram embeddings, not just whole-word
- Shared subword parameters across many words
- Typo resilience as a side benefit

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

$\mathbf{v}_w = \sum_{g \in \mathcal{G}_w} \mathbf{z}_g$ means a rare word's vector is built from n-gram embeddings that are shared — and therefore well-trained — across many other words in the vocabulary, not learned from that single word's scarce occurrences alone.

Production Perspective & Trade-offs

This same subword-sharing property makes FastText resilient to typos and OOV terms, which is valuable for noisy, user-generated production text — at the cost of a substantially larger memory footprint, since it must store embeddings for the full n-gram hash-bucket table, not just the word vocabulary.

Common Mistakes

1. Assuming FastText is as computationally slow as a context-dependent transformer model — it's still a static lookup-and-sum model, much cheaper than a forward pass through an encoder.
2. Forgetting the memory trade-off: FastText's rare-word quality gain comes at the cost of a much larger stored model than Word2Vec.

COMMON FOLLOW-UP QUESTIONS

> **Q: Does FastText require more memory than Word2Vec?**
>
> Yes — it must store embeddings for both whole words and the full set of character n-gram hash buckets, which is substantially larger than Word2Vec's single per-word table.

> **Q: Why does subword sharing specifically help rare words rather than common ones?**
>
> Common words already get plenty of direct training signal on their own; rare words benefit disproportionately because their vector is bootstrapped from n-grams that other, more frequent words have already trained well.

One-Line Takeaway

> **Takeaway:** FastText's character n-gram sharing lets rare words borrow well-trained subword signal from common words, at the cost of a much larger stored model than Word2Vec.

Question 43: Why are bidirectional RNNs unsuitable for autoregressive text generation?

[ESSENTIAL]

Conversational Answer

Autoregressive generation produces text one token at a time, using only what's been generated so far. A bidirectional RNN's backward pass requires the *entire* sequence to already exist to compute backward hidden states — which is structurally impossible when you're still in the middle of generating that sequence.

Intuitive Example

To compute a bidirectional RNN's backward hidden state at position 3, you need to already know positions 4, 5, 6... but during generation those tokens don't exist yet — they're exactly what you're trying to produce.

Key Interview Points

- Autoregressive generation only has access to past tokens
- Bidirectional models require the full sequence upfront
- Causal masking is the correct mechanism for generation instead

---

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

No formula is needed — the incompatibility is structural: bidirectional processing requires the complete sequence as input, while autoregressive generation produces that sequence incrementally, one token dependent only on the past.

Production Perspective & Trade-offs

For generation tasks, causal (unidirectional) decoder architectures with causal masking are used instead, guaranteeing that each position's computation only ever depends on earlier positions — this is what makes step-by-step token generation well-defined at all.

Common Mistakes

1. Proposing a bidirectional model (like BERT) directly for a text-generation task.
2. Confusing "the model was trained on full sequences" with "the model can generate token-by-token" — training-time access to full sequences (via causal masking, not bidirectionality) is different from requiring the full sequence at inference.

COMMON FOLLOW-UP QUESTIONS

Q: Where are bidirectional models actually useful, then?

Sequence tagging (NER, POS) and representation learning (BERT-style embeddings) — any task where the entire input sequence is already available at inference time and there's no token-by-token generation involved.

Q: How does causal masking enforce the autoregressive constraint?

It masks out (sets to $-\infty$ before softmax) attention scores from any position to future positions, so each token's representation is mathematically guaranteed to only depend on itself and earlier tokens.

One-Line Takeaway

> **Takeaway:** Bidirectional models need the full sequence to exist upfront; autoregressive generation produces that sequence incrementally — the two are structurally incompatible.

Question 44: How does teacher forcing affect Seq2Seq training?

[ESSENTIAL]

Conversational Answer

Teacher forcing feeds the ground-truth target tokens into the decoder during training, instead of the model's own (possibly wrong) previous predictions. This stabilizes and speeds up early training by preventing one early mistake from cascading through the rest of the sequence — but it also means the model never practices recovering from its own errors.

Intuitive Example

If a translation model incorrectly predicts word 3 of 10 during training, teacher forcing still feeds it the *correct* word 3 as input to predict word 4 — so a single early mistake doesn't compound into a completely garbled rest-of-sentence during training.

Key Interview Points

- Ground-truth tokens fed as decoder input during training
- Speeds up and stabilizes early training
- Introduces exposure bias (see Question 46)

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

No formula is required — teacher forcing is a training procedure choice (what to feed as decoder input), not a computed quantity.

Production Perspective & Trade-offs

Teacher forcing's speed and stability during training come at a direct cost at inference time: since the model never trains on its own imperfect outputs, it can be brittle when it inevitably has to condition on its own predictions in production. Scheduled sampling — gradually mixing in the model's own predictions during training — is the standard mitigation.

Common Mistakes

1. Using teacher forcing during inference, where ground-truth targets don't exist by definition.
2. Not anticipating exposure bias as a known consequence of pure teacher forcing, and being surprised when generation quality degrades on longer outputs in production.

COMMON FOLLOW-UP QUESTIONS

Q: How do you mitigate the exposure bias teacher forcing introduces?

Scheduled sampling — gradually transition from feeding ground-truth tokens to feeding the model's own predictions as training progresses, so the model gets practice recovering from its own errors before deployment.

Q: Why does teacher forcing speed up convergence specifically?

Because the decoder always conditions on the correct prefix, gradient signal at each step reflects a clean, consistent target rather than being corrupted by compounding earlier prediction errors.

One-Line Takeaway

> **Takeaway:** Teacher forcing trades training speed and stability for a model that's never practiced recovering from its own mistakes — exposure bias is the direct cost.

Question 45: Explain greedy decoding versus beam search.

[ESSENTIAL]

Conversational Answer

Greedy decoding picks the single most likely token at every step and never looks back — fast, but it can lock in a locally-good choice that leads to a globally worse sequence. Beam search keeps track of the top-B most likely partial sequences at each step, exploring more of the space at the cost of extra compute.

Intuitive Example

Greedy decoding is like always taking the next best-looking turn on a road trip without considering where it leads — beam search is keeping the top few plausible routes in mind simultaneously before committing.

Key Interview Points

- Greedy: local, single-path, $O(L)$
- Beam search: multi-path, $O(L \cdot B)$
- Beam width B trades quality for latency/memory

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Greedy decoding selects $y_t = \arg\max P(y \mid y_{<t})$ at each step independently; beam search instead maintains the top-B scoring partial sequences at every step, expanding and re-pruning at each position — a heuristic search, not a guarantee of the globally optimal sequence.

	Greedy	Beam Search
Paths tracked	1	B
Time complexity	$O(L)$	$O(L \cdot B)$
Optimality	Locally optimal only	Better, still not globally guaranteed
Typical production width	N/A	2-5

Production Perspective & Trade-offs

Larger beam widths ($B > 10$) noticeably increase both latency and memory in production, so a beam size of 2-5 is a common practical balance between output quality and serving cost. Length normalization is typically required in beam search, since multiplying more probabilities (longer sequences) otherwise systematically favors shorter outputs.

Common Mistakes

1. Believing beam search finds the provably globally optimal sequence — it's still a heuristic, bounded search, not exhaustive.
2. Forgetting length normalization, which causes beam search to systematically prefer shorter (and often worse) completions.

COMMON FOLLOW-UP QUESTIONS

> **Q: Why apply length normalization in beam search?**
> Without it, multiplying more conditional probabilities together (a longer sequence) always yields a smaller joint probability, so beam search would systematically favor shorter — not necessarily better — completions.

> **Q: When is greedy decoding actually the right choice over beam search?**
> When latency is the dominant constraint and the task is simple enough that locally-optimal choices rarely diverge from a good global sequence — e.g. short, low-ambiguity completions.

One-Line Takeaway

> **Takeaway:** Greedy decoding is fast but locally short-sighted; beam search trades compute for exploring more candidate sequences, without ever guaranteeing the true global optimum.

Question 46: What is exposure bias in Seq2Seq models?

[ESSENTIAL]

Conversational Answer

Exposure bias is the mismatch between how a model is trained (teacher forcing, always conditioned on correct ground-truth tokens) and how it's used at inference (conditioned on its own, possibly imperfect predictions). Early errors during inference can compound in a way the model never practiced handling during training.

Intuitive Example

A model that only ever practiced summarizing perfectly-formed reference text during training can start generating increasingly repetitive or incoherent output partway through a long generation at inference time, once it's a few tokens into its own imperfect output and has never seen that situation before.

Key Interview Points

- Training-inference input distribution mismatch
- Error propagation compounds during inference
- Scheduled sampling is the standard mitigation

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

No formula is required — exposure bias is a distributional mismatch between the decoder's training-time inputs (always ground truth) and inference-time inputs (its own predictions), not a computed quantity.

Production Perspective & Trade-offs

In production, exposure bias tends to show up specifically on longer generations, where small early errors have more opportunity to compound into repetitive or off-topic output — this is part of why generation quality often degrades noticeably as output length grows.

Common Mistakes

1. Attributing exposure-bias symptoms to a training-data quality issue rather than recognizing it as a training-inference procedural mismatch.
2. Not testing generation quality specifically on long outputs, where exposure bias effects are most visible.

COMMON FOLLOW-UP QUESTIONS

> **Q: How does RLHF help address exposure bias?**
>
> RLHF optimizes the model based on the quality of its own complete generated sequences (via a reward signal), which directly aligns training-time optimization with inference-time behavior, rather than only ever training against ground-truth prefixes.

> **Q: Is exposure bias unique to RNN-based Seq2Seq models?**
>
> No — it affects any autoregressive model trained with teacher forcing, including Transformer-based decoders, since the core mismatch is about training vs. inference conditioning, not the specific architecture.

One-Line Takeaway

> **Takeaway:** Exposure bias is the gap between training on perfect ground-truth prefixes and inference on the model's own imperfect predictions — and it gets worse the longer the generation.

Question 47: Why is perplexity not always a good evaluation metric?

[ESSENTIAL]

Conversational Answer

Perplexity only measures how well a model predicts the next token statistically — it says nothing about whether the output is factually correct, logically consistent, or actually useful. A model can produce grammatically fluent nonsense and still score a low (good-looking) perplexity.

Intuitive Example

A model could generate a perfectly fluent but completely fabricated news summary and still achieve low perplexity, because perplexity only checks "was this a plausible next word," never "is this true."

Key Interview Points

- Measures next-token prediction confidence only
- No notion of factual or semantic correctness
- Tokenizer-dependent, so not comparable across models

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Perplexity is purely a function of the model's assigned probability to the actual next tokens in a test set — it has no mechanism for evaluating truthfulness, coherence beyond the local window, or task usefulness, and (as covered in Question 18) it isn't even comparable across models with different tokenizers.

Production Perspective & Trade-offs

Because perplexity is tokenizer-dependent, it can't be used to compare models across different tokenizer configurations — this makes it useful mainly as an internal training-progress signal for a single fixed model/tokenizer pair, not a general-purpose quality benchmark.

Common Mistakes

1. Comparing perplexity scores between models that use different subword tokenizers — the comparison is not meaningful.
2. Using perplexity alone as a proxy for "generation quality" in a product sense, when it says nothing about factual correctness or usefulness.

COMMON FOLLOW-UP QUESTIONS

> **Q: What metrics evaluate summarization quality better than perplexity?**
>
> BERTScore for semantic alignment, or LLM-as-a-Judge approaches that explicitly assess factual consistency and coherence — both go beyond next-token prediction confidence.

> **Q: Is there any setting where perplexity alone is sufficient?**
>
> As an internal signal for tracking whether a single model is still improving during training on a fixed dataset and tokenizer — it's much less useful for cross-model or product-quality comparisons.

One-Line Takeaway

> **Takeaway:** Perplexity measures next-token confidence, not truth or usefulness — a fluent, low-perplexity model can still be factually wrong.

Question 48: How does Byte Pair Encoding (BPE) differ from WordPiece and Unigram Language Models?

[ESSENTIAL]

Conversational Answer

BPE is bottom-up: it repeatedly merges the most *frequent* adjacent pair of symbols. WordPiece is also bottom-up but merges the pair that most *increases corpus likelihood* rather than raw frequency. The Unigram Language Model tokenizer works top-down instead — starting from a large vocabulary and pruning tokens that contribute least to corpus likelihood.

Intuitive Example

Given many occurrences of "un" and "able" appearing together, BPE would merge them purely because the pair is frequent; WordPiece would only merge them if doing so improves the model's overall likelihood score more than other frequent candidate pairs would.

Key Interview Points

BPE: bottom-up, frequency-driven merges

- WordPiece: bottom-up, likelihood-driven merges
- Unigram LM: top-down, likelihood-based pruning

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

$$\text{Score}(a, b) = \frac{\text{Count}(ab)}{\text{Count}(a) \times \text{Count}(b)}$$

This is WordPiece's merge score — it favors pairs that co-occur *more than their individual frequencies alone would predict*, unlike BPE which just picks the single highest raw pair count. Unigram LM instead starts from a large candidate vocabulary and iteratively removes tokens whose removal costs the least corpus likelihood.

	BPE	WordPiece	Unigram LM
Direction	Bottom-up	Bottom-up	Top-down
Merge/prune criterion	Raw pair frequency	Likelihood-ratio score	Likelihood contribution
Used by	GPT family	BERT	SentencePiece (T5, ALBERT)

Production Perspective & Trade-offs

Unigram LM tokenizers tend to offer more flexible, probabilistic subword segmentations (multiple valid tokenizations can be sampled), which has shown benefits for multilingual model performance compared to the single deterministic segmentation BPE/WordPiece produce.

Common Mistakes

1. Assuming all subword tokenizers use the same merge/prune criterion — the underlying scoring functions are genuinely different.
2. Not knowing which major model families use which tokenizer (a common quick-fire follow-up).

COMMON FOLLOW-UP QUESTIONS

> **Q: Which tokenizer does BERT use, and which does GPT use?**
>
> BERT uses WordPiece; GPT models use BPE.

> **Q: Why might Unigram LM's probabilistic segmentation help multilingual models specifically?**
>
> Different languages have very different morphological structures, and allowing multiple valid tokenizations (rather than one fixed deterministic split) gives the model more flexibility to represent varied morphology well across languages.

One-Line Takeaway

> **Takeaway:** BPE merges by raw frequency, WordPiece merges by likelihood ratio, and Unigram LM prunes top-down by likelihood contribution — three different criteria for the same underlying goal.

Question 49: Why are contextual embeddings superior to static embeddings?

[ESSENTIAL]

Conversational Answer

Contextual embeddings compute a fresh vector for each occurrence of a word based on its surrounding sentence, resolving polysemy that static embeddings structurally cannot — "bank" gets a different vector in "river bank" versus "money bank," which a static lookup table can never do.

Intuitive Example

Static Word2Vec gives "bank" one vector, period. BERT gives "bank" in "I sat by the river bank" a noticeably different vector than "bank" in "I deposited money at the bank," because the surrounding words directly shape each computation.

Key Interview Points

- Polysemy resolution via context
- Dynamic, per-occurrence vector computation
- Same underlying superiority discussed in Question 10

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

As covered in Question 10, this comes down to self-attention letting every token's output representation be a function of every other token in its specific sequence — a strictly more expressive computation than a fixed per-word table lookup.

Production Perspective & Trade-offs

That expressiveness isn't free: contextual embeddings require a full transformer forward pass per input, which is meaningfully more expensive to serve than a static embedding table lookup — the same cost/accuracy trade-off discussed in Question 10 applies here directly.

Common Mistakes

1. Assuming static embeddings are now obsolete entirely — they remain the right choice for low-latency similarity search where a transformer forward pass isn't affordable.
2. Not recognizing that this question and Question 10 are asking about the same underlying trade-off from different angles.

COMMON FOLLOW-UP QUESTIONS

> **Q: How does BERT construct a contextual embedding, concretely?**
>
> By passing token embeddings through multiple self-attention layers, where each layer updates every token's representation based on all other tokens in the sequence, progressively building richer, context-aware representations.

> **Q: Is there a middle ground between fully static and fully contextual embeddings?**
>
> Yes — pre-computing and caching contextual embeddings for frequently-seen fixed phrases is a common production compromise, trading some freshness/dynamism for lookup speed on high-traffic queries.

One-Line Takeaway

> **Takeaway:** Contextual embeddings resolve polysemy by computing a fresh, sentence-aware vector per occurrence — the same expressiveness-vs-cost trade-off as Question 10, viewed from the "why" angle.

Question 50: What are the computational complexity differences between RNNs and self-attention?

[ESSENTIAL]

Conversational Answer

RNNs process sequentially — $O(L \cdot d^2)$ time overall, with information taking $O(L)$ sequential steps to travel between distant tokens. Self-attention processes the whole sequence in parallel — $O(L^2 \cdot d)$ time and memory — but any two tokens are directly connected in a single step, $O(1)$ path length.

Intuitive Example

For a 1,000-token document, an RNN needs up to 1,000 sequential steps for information to flow from the first token to the last; self-attention connects them directly in one computation, at the cost of computing and storing a 1,000×1,000 attention score matrix.

Key Interview Points

- RNN: $O(L \cdot d^2)$ time, sequential, $O(L)$ path length
- Self-attention: $O(L^2 \cdot d)$ time/memory, parallel, $O(1)$ path length
- The crossover point depends on both L and d

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

$O(L \cdot d^2)$ (RNN, sequential) vs. $O(L^2 \cdot d)$ (self-attention, parallel) — these are genuinely different complexity classes, not just different constants, so which one is "cheaper" flips depending on whether L or d dominates for a given model and sequence length.

Production Perspective & Trade-offs

Self-attention's parallelism makes training dramatically faster on GPUs, but its quadratic $O(L^2)$ memory cost makes serving very long sequences expensive — this is precisely the motivation behind efficient-attention variants (sparse attention, sliding-window attention, FlashAttention) used in production long-context models.

Common Mistakes

1. Assuming self-attention is unconditionally more efficient than RNNs — it's only more efficient during training due to parallelization; at long sequence lengths its quadratic memory cost can dominate.

2. Forgetting that the RNN-vs-attention crossover point depends on both sequence length L and hidden dimension d, not L alone.

COMMON FOLLOW-UP QUESTIONS

> **Q: At roughly what sequence length does self-attention's compute cost exceed an RNN's?**
>
> Once sequence length L grows large enough relative to hidden dimension d that the $O(L^2 \cdot d)$ term outweighs the RNN's $O(L \cdot d^2)$ term — practically, this tends to matter most for very long-context serving workloads.

> **Q: What production techniques address self-attention's quadratic memory cost?**
>
> Sparse attention patterns, sliding-window attention, and kernel-fusion approaches like FlashAttention all reduce the effective memory and compute cost of long-context self-attention without abandoning its parallelism benefits.

One-Line Takeaway

> **Takeaway:** RNNs and self-attention sit on opposite ends of the same trade-off — sequential-but-linear-memory vs. parallel-but-quadratic-memory — and the better choice depends on sequence length relative to hidden dimension.

Classical NLP Foundations Interview Cheatsheet: Final Revision Sheet

1. Quick-Recall One-Line Takeaways Table

#	Question	One-Line Takeaway
1	What is NLP, and what are the major NLP tasks?	NLP is a task taxonomy (classification, tagging, generation, QA); task shape determines architecture and latency budget.
2	Explain a typical NLP pipeline from raw text to prediction.	The pipeline is a chain of shape transformations, and it breaks silently when preprocessing drifts between training and serving.
3	What is the difference between stemming and lemmatization?	Stemming trades correctness for speed via suffix rules; lemmatization trades speed for a real dictionary root.

#	Question	One-Line Takeaway
4	Why is tokenization necessary?	Tokenization trades vocabulary size against sequence length, rippling into every downstream memory and compute cost.
5	Compare character, word, and subword tokenization.	Character minimizes vocabulary at the cost of sequence length; word does the reverse; subword is the balance point.
6	Why did NLP move from word to subword tokenization?	Subword tokenization caps vocabulary size while still statistically decomposing any unseen word.
7	What are OOV words, and how can they be handled?	The practical question is whether your OOV fallback discards signal or preserves it.
8	Explain the Distributional Hypothesis.	"Context determines meaning" is the unlabeled-data assumption behind every embedding method.
9	Why do sparse representations fail to capture similarity?	Sparse representations are semantically blind by construction — every word is orthogonal to every other.
10	Compare static embeddings and contextual embeddings.	Static trades context-sensitivity for $O(1)$ speed; contextual trades inference cost for polysemy resolution.
11	Why were RNNs introduced after Bag-of-Words/TF-IDF?	RNNs recover word order, which count-based methods discard by design, at the cost of sequential architecture.
12	Why were Transformers able to replace RNNs?	Transformers trade recurrence's cheap-but-sequential path length for attention's parallel-but-quadratic path length.
13	Derive TF-IDF and compute scores for a small corpus.	TF-IDF rewards terms frequent locally but rare globally; L_2 normalization makes vectors length-comparable.
14	Compute cosine similarity between two TF-IDF vectors.	Cosine similarity measures directional alignment, not magnitude.
15	Derive N-gram sentence probability via the chain rule.	The Markov assumption is what makes estimating sequence probability from finite data tractable.

#	Question	One-Line Takeaway
16	Explain MLE for N-gram language models.	MLE is a corpus count ratio — correct on what it's seen, undefined on what it hasn't.
17	Why is Laplace smoothing required?	Laplace smoothing trades validity for crude uniform redistribution, which Kneser-Ney improves on at scale.
18	What is Perplexity?	Perplexity is exponentiated cross-entropy, only comparable between models sharing an identical tokenizer.
19	Explain CBOW and Skip-gram.	CBOW predicts target from averaged context (fast, weaker on rare words); Skip-gram is the reverse.
20	Why does Negative Sampling speed up Word2Vec?	Negative sampling replaces $O(\|V\|)$ softmax with $O(k)$ binary classification.
21	Why do RNNs suffer from vanishing gradients?	Repeated multiplication by the recurrent weight matrix during BPTT decays the gradient if its spectral radius is below 1.
22	How does LSTM's cell state mitigate vanishing gradients?	An additive cell-state pathway gated by the forget gate — protection that holds only while the gate stays open.
23	What challenges arise deploying NLP models to production?	Production challenges are mostly about consistency and cost under load, not raw model accuracy.
24	How do you handle vocabulary drift in production?	Byte-fallback prevents hard failures; scheduled retraining closes the representation gap.
25	What is training-serving skew?	A silent accuracy killer from any divergence between training-time and serving-time preprocessing.
26	Explain Data Drift vs. Concept Drift.	Data drift changes what inputs look like; concept drift changes what they mean.
27	When do you choose batch vs. real-time inference?	The workload's actual latency SLA should decide, not preference.
28	How do quantization and pruning reduce inference cost?	Quantization reliably shrinks VRAM; pruning's speedup is conditional on sparse-op hardware support.

#	Question	One-Line Takeaway
29	What preprocessing inconsistencies degrade performance?	Automated parity testing between training and serving pipelines is the fix, not code review alone.
30	What monitoring metrics matter in production?	Systems, data, and model-quality signals are needed independently — systems health alone can look fine while quality degrades.
31	Why can BLEU/ROUGE give misleading results?	They measure textual overlap, not meaning — fooled by both synonyms and negations.
32	When do you prefer BERTScore over BLEU?	BERTScore catches semantic equivalence BLEU can't, at the cost of a transformer forward pass per comparison.
33	How do you debug poor precision but high recall?	Try threshold tuning first — immediate and free — before touching the model itself.
34	How do you diagnose exploding gradients?	A loss spike to NaN with large gradient norms is the signature; gradient clipping is the direct fix.
35	How do you perform error analysis for an NLP system?	Manual, categorized error analysis surfaces fixable bugs that aggregate metrics hide.
36	Design a spam classifier for millions of users.	A tiered cheap-filter-then-expensive-model design is what makes web scale affordable.
37	Design an autocomplete/search suggestion system.	An in-memory trie ranked by smoothed frequency makes sub-10ms autocomplete possible.
38	Design a sentiment pipeline for social media.	Preprocessing should treat emojis/punctuation as signal, and drift monitoring needs a tighter cadence.
39	Design an FAQ chatbot without LLMs.	BM25 keyword matching is fast and deterministic, but can't bridge a vocabulary gap without a semantic layer.
40	How do you choose between TF-IDF, Word2Vec, FastText, LSTM, Transformer?	Start from the cheapest viable baseline and upgrade only when the accuracy gain justifies the cost.

#	Question	One-Line Takeaway
41	Why is BM25 better than TF-IDF for retrieval?	BM25 caps term-frequency dominance and corrects for document length, without needing model inference.
42	Why does FastText outperform Word2Vec on rare words?	Rare words borrow well-trained subword signal from common words, at the cost of a larger stored model.
43	Why are bidirectional RNNs unsuitable for generation?	Bidirectional models need the full sequence upfront; generation produces it incrementally.
44	How does teacher forcing affect Seq2Seq training?	Teacher forcing trades training speed/stability for a model that's never practiced recovering from its own mistakes.
45	Explain greedy decoding vs. beam search.	Greedy is fast but locally short-sighted; beam search explores more candidates without guaranteeing the optimum.
46	What is exposure bias in Seq2Seq models?	The gap between training on perfect prefixes and inference on the model's own imperfect predictions.
47	Why is perplexity not always a good evaluation metric?	It measures next-token confidence, not truth or usefulness.
48	How does BPE differ from WordPiece and Unigram LM?	BPE merges by frequency, WordPiece by likelihood ratio, Unigram LM prunes top-down by likelihood.
49	Why are contextual embeddings superior to static ones?	They resolve polysemy by computing a fresh, context-aware vector per occurrence.
50	What are the complexity differences between RNNs and self-attention?	Sequential-but-linear-memory vs. parallel-but-quadratic-memory — the better choice depends on L relative to d.

2. Essential Formula Cheat Sheet

- **Smoothed IDF**: $\mathrm{idf}_t = \ln\left(\frac{1+N}{1+\mathrm{df}_t}\right) + 1$
- **TF-IDF Score**: $\text{tf-idf}_{t,d} = \mathrm{tf}_{t,d} \times \mathrm{idf}_t$
- **Cosine Similarity**: $\mathrm{CosineSim}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u}\cdot\mathbf{v}}{\|\mathbf{u}\|_2\|\mathbf{v}\|_2}$

- **BM25 Score**: $\text{Score}(D, Q) = \sum_i \text{IDF}(q_i) \cdot \frac{f(q_i, D)(k_1+1)}{f(q_i, D) + k_1(1 - b + b\frac{|D|}{\text{avgdl}})}$
- **Chain Rule of Probability**: $P(w_1, \ldots, w_L) = \prod_i P(w_i \mid w_{<i})$
- **Laplace Smoothing**: $P_{\text{Laplace}}(w_i \mid w_{i-1}) = \frac{C(w_{i-1}, w_i)+1}{C(w_{i-1})+|V|}$
- **Perplexity**: $\text{PPL}(W) = e^{\mathcal{L}_{\text{CE}}} = P(W)^{-1/m}$
- **Negative Sampling Loss**: $\mathcal{L}_{\text{SGNS}} = -\ln \sigma(\mathbf{v}_w \cdot \mathbf{v}'_{w_c}) - \sum_{i=1}^{k} \ln \sigma(-\mathbf{v}_w \cdot \mathbf{v}'_{w_i})$
- **RNN Hidden State Update**: $\mathbf{h}_t = \tanh(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t + \mathbf{b}_h)$
- **LSTM Cell State Update**: $\mathbf{C}_t = \mathbf{f}_t \odot \mathbf{C}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{C}}_t$
- **BLEU**: $\text{BLEU} = \text{BP} \times \exp\left(\sum_n w_n \log p_n\right)$
- **Word Error Rate**: $\text{WER} = \frac{S+D+I}{N}$
- **F1-Score**: $\text{F1} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$
- **Population Stability Index**: $\text{PSI} = \sum_k (\text{Actual}_k\% - \text{Expected}_k\%) \times \ln\left(\frac{\text{Actual}_k\%}{\text{Expected}_k\%}\right)$

3. Top Follow-up Q&As

1. **Q: Can static embeddings resolve part-of-speech ambiguity?** → No — the vector is fixed regardless of grammatical role.
2. **Q: What is a reasonable value for k in negative sampling?** → 5-20 for small datasets, 2-5 for large corpora.
3. **Q: Why does gradient clipping not fix vanishing gradients?** → It only bounds gradients that are too large; a decayed-to-zero gradient has nothing left to clip.
4. **Q: Why is comparing perplexity across tokenizers invalid?** → A smaller vocabulary structurally yields lower perplexity regardless of actual model quality.
5. **Q: What's the fastest fix for high recall but low precision?** → Raise the decision threshold — immediate, free, no retraining required.
6. **Q: Why do Transformers need positional encodings?** → Self-attention is permutation-invariant by default and has no other way to encode order.
7. **Q: What's the standard mitigation for exposure bias?** → Scheduled sampling — gradually shift from ground-truth to model-generated inputs during training.
8. **Q: Which tokenizer does BERT use vs. GPT?** → BERT uses WordPiece; GPT uses BPE.